

**IMPLEMENTASI MONITORING KESEHATAN JARINGAN
BERBASIS MIKROTIK DENGAN PENDEKATAN RULE BASED DAN
EVENT CORRELATION**

TUGAS AKHIR



Disusun Oleh :
MUHAMAD TSANI PUTRA TRONCHET
123220115

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

IMPLEMENTASI MONITORING KESEHATAN JARINGAN BERBASIS MIKROTIK DENGAN PENDEKATAN RULE BASED DAN EVENT CORRELATION

TUGAS AKHIR

Tugas Akhir ini sebagai salah satu syarat untuk memperoleh gelar sarjana Informatika
Universitas Pembangunan Nasional “Veteran” Yogyakarta



Disusun Oleh :
MUHAMAD TSANI PUTRA TRONCHET
123220115

PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN”
YOGYAKARTA
2026

HALAMAN PENGESAHAN PEMBIMBING

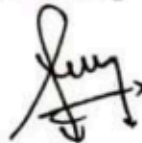
IMPLEMENTASI MONITORING KESEHATAN JARINGAN BERBASIS MIKROTIK DENGAN PENDEKATAN RULE BASED DAN EVENT CORRELATION

Disusun Oleh:
Muhamad Tsani Putra Tronchet
123220115

Telah diuji dan dinyatakan lulus oleh pembimbing
pada tanggal: 11 Mei 2026

Menyetujui,

Pembimbing



Andrey Ferriyan, S.T., M.Cs., Ph.D.
NUPTK. 1255761662130193

Mengetahui,

Koordinator Program Studi



Dessyanto Boedi Prasetyo, S.T., M.T.
NIDN. 0505127501

HALAMAN PENGESAHAN PENGUJI

IMPLEMENTASI MONITORING KESEHATAN JARINGAN BERBASIS MIKROTIK DENGAN PENDEKATAN RULE BASED DAN EVENT CORRELATION

Disusun Oleh:

Muhamad Tsani Putra Tronchet

123220115

Telah diuji dan dinyatakan lulus oleh penguji

pada tanggal: 11 Mei 2026

Menyetujui,

Penguji 1



Andrey Ferriyan, S.T., M.Cs., Ph.D.
NUPTK. 1255761662130193

Penguji 2



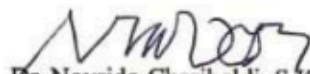
Bagus Muhammad Akbar, S.S.T., M.Kom.
NIDN. 0001088905

Penguji 3



Rudy Cahyadi, S.Si., M.T.
NIDN. 0019037504

Penguji 4



Dr. Novrido Charibaldi, S.Kom., M.Kom.
NIDN. 0503116801

SURAT PERNYATAAN KARYA ASLI TUGAS AKHIR

Sebagai mahasiswa Program Studi Informatika Fakultas Teknik Industri Universitas Pembangunan Nasional "Veteran" Yogyakarta, yang bertanda tangan di bawah ini, saya:

Nama : Muhamad Tsani Putra Tronchet
NIM : 123220115

Menyatakan bahwa karya ilmiah saya yang berjudul:

Implementasi Monitoring Kesehatan Jaringan Berbasis MikroTik dengan Pendekatan Rule Based dan Event Correlation

Merupakan karya asli saya dan belum pernah dipublikasikan dimanapun. Apabila di kemudian hari, karya saya disinyalir bukan merupakan karya asli saya, maka saya bersedia menerima konsekuensi apa pun yang diberikan Program Studi Informatika Fakultas Teknik Industri Universitas Pembangunan Nasional "Veteran" Yogyakarta kepada saya.

Demikian surat pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Yogyakarta
Pada Tanggal : 8 Juni 2026

Yang Menyatakan



Muhamad Tsani Putra Tronchet
NIM. 123220115

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini:

Nama : Muhamad Tsani Putra Tronchet
NIM : 123220115
Prodi : Informatika

Dengan ini saya menyatakan bahwa judul Tugas Akhir
**Implementasi Monitoring Kesehatan Jaringan Berbasis MikroTik dengan Pendekatan
Rule Based dan Event Correlation**

Adalah hasil kerja saya dan benar bebas dari plagiasi kecuali cuplikan serta ringkasan yang terdapat di dalamnya telah saya jelaskan sumbernya (Sitasi) dengan jelas. Apabila pernyataan ini terbukti tidak benar maka saya bersedia menerima sanksi sesuai peraturan Mendiknas RI No 17 Tahun 2010 dan Peraturan Perundang-undangan yang berlaku.

Demikian surat pernyataan ini saya buat dengan penuh tanggung jawab.

Dibuat di : Yogyakarta

Pada Tanggal : 8 Juni 2025

Yang Menyatakan



Muhamad Tsani Putra Tronchet
NIM. 123220115

ABSTRAK

Perkembangan teknologi informasi meningkatkan kebutuhan akan sistem monitoring jaringan yang mampu mendeteksi gangguan secara cepat dan memberikan informasi yang lebih kontekstual. Monitoring berbasis MikroTik umumnya masih berfokus pada pemantauan parameter dan notifikasi perubahan kondisi secara terpisah, sehingga analisis keterkaitan antar kejadian masih dilakukan secara manual. Penelitian ini mengusulkan dan mengevaluasi sistem monitoring kesehatan jaringan berbasis MikroTik dengan pendekatan *rule-based detection* dan *event correlation* untuk menghasilkan indikasi awal gangguan jaringan berdasarkan keterkaitan antar *event*.

Penelitian dilakukan pada lingkungan simulasi GNS3 menggunakan MikroTik Cloud Hosted Router (CHR). Data kesehatan jaringan berupa penggunaan CPU, memori, *delay*, *packet loss*, dan status *interface* dikumpulkan secara periodik melalui *RouterOS API*. Mekanisme *rule-based detection* digunakan untuk membentuk *event* berdasarkan nilai *threshold* dan durasi pelanggaran melalui *sliding window*, disertai mekanisme *event lifecycle* untuk memastikan konsistensi status *event*. Selanjutnya, *event correlation* digunakan untuk mengaitkan beberapa *event* yang terjadi dalam rentang waktu tertentu berdasarkan kedekatan temporal dan keterkaitan parameter. Evaluasi dilakukan menggunakan metrik *Detection Rate* (DR) dan *Mean Time to Detect* (MTTD).

Hasil penelitian menunjukkan bahwa sistem berhasil membentuk *event* gangguan dan mengaitkannya menjadi dua jenis indikasi, yaitu degradasi kualitas jaringan tanpa keterlibatan sumber daya perangkat serta degradasi kualitas jaringan akibat keterbatasan sumber daya perangkat. Berdasarkan 18 skenario insiden yang diuji, sistem memperoleh nilai *Detection Rate* sebesar 100% dan *Mean Time to Detect* sebesar 50,5 detik, di mana nilai MTTD tersebut dipengaruhi oleh durasi pelanggaran *threshold* selama 60 detik pada mekanisme *sliding window*. Hasil tersebut menunjukkan bahwa pendekatan *rule-based detection* dan *event correlation* mampu mendeteksi seluruh insiden yang diuji serta menghasilkan indikasi gangguan yang lebih terstruktur dibandingkan monitoring berbasis kejadian tunggal.

Kata Kunci: *monitoring jaringan, MikroTik, rule-based, event correlation, event lifecycle, kesehatan jaringan.*

ABSTRACT

The advancement of information technology has increased the need for network monitoring systems capable of rapidly detecting disruptions and providing more contextual information. MikroTik-based monitoring generally focuses on monitoring network parameters and notifying condition changes separately, causing the analysis of relationships between events to still be performed manually. This study proposes and evaluates a MikroTik-based network health monitoring system using rule-based detection and event correlation approaches to generate early indications of network disruptions based on the relationships among events.

The study was conducted in a GNS3 simulation environment using MikroTik Cloud Hosted Router (CHR). Network health data, including CPU usage, memory usage, delay, packet loss, and interface status, were collected periodically through the RouterOS API. The rule-based detection mechanism was used to generate events based on threshold values and violation durations using a sliding window approach, accompanied by an event lifecycle mechanism to ensure event status consistency. Furthermore, event correlation was applied to associate multiple events occurring within a specific time range based on temporal proximity and parameter relationships. The evaluation was performed using Detection Rate (DR) and Mean Time to Detect (MTTD) metrics.

The results show that the system successfully generated disruption events and correlated them into two types of indications: network quality degradation without device resource involvement and network quality degradation caused by device resource limitations. Based on 18 tested incident scenarios, the system achieved a Detection Rate of 100% and a Mean Time to Detect of 50.5 seconds, where the MTTD value was influenced by the 60-second threshold violation duration configured in the sliding window mechanism. These findings indicate that the rule-based detection and event correlation approaches were able to detect all tested incidents and provide more structured disruption indications compared to single-event-based monitoring.

Keywords: *network monitoring, MikroTik, rule-based, event correlation, event lifecycle, network health.*

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat dan hidayah-Nya, sehingga penulis dapat menyelesaikan skripsi yang berjudul “Implementasi Monitoring Kesehatan Jaringan Berbasis Mikrotik dengan Pendekatan Rule-Based dan Event Correlation” dengan lancar. Penyusunan skripsi ini dilakukan sebagai salah satu syarat untuk memperoleh gelar Sarjana pada Program Studi Teknik Informatika Universitas Pembangunan Nasional “Veteran” Yogyakarta.

Penulis menyadari bahwa proses penyusunan skripsi ini tidak terlepas dari berbagai kendala. Namun, berkat bimbingan, bantuan, serta dukungan dari berbagai pihak, skripsi ini dapat diselesaikan dengan baik. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan rasa terima kasih yang sebesar-besarnya kepada seluruh pihak yang telah memberikan kontribusi, baik secara langsung maupun tidak langsung, yaitu kepada:

1. Allah SWT yang telah memudahkan dan melancarkan segala urusan sehingga penulis dapat sampai pada tahap ini.
2. Kedua orang tua penulis, Bapak Don't Rudolph Tronchet dan Ibu Fajar Indah Tridestina, atas kasih sayang, doa, motivasi, serta nasihat yang tidak pernah putus. Dukungan dari beliau berdua menjadi sumber kekuatan utama bagi penulis hingga karya ini dapat terselesaikan dengan baik.
3. Bapak Andrey Ferriyan, S.T., M.Cs., Ph.D. selaku dosen pembimbing yang telah memberikan arahan, bimbingan, serta masukan yang sangat berharga selama proses penyusunan skripsi ini.
4. Bapak Bagus Muhammad Akbar, S.S.T., M.Kom., Bapak Rudy Cahyadi, S.Si., M.T., dan Dr. Novrido Charibaldi, S.Kom., M.Kom. selaku dosen penguji yang telah memberikan saran serta bimbingan dalam menyelesaikan tugas akhir ini.
5. Seorang yang telah mendampingi penulis sejak bangku Sekolah Menengah Atas, yang senantiasa memberikan dukungan, doa, dan semangat, serta menemani penulis dalam perjalanan perkuliahan hingga penyusunan skripsi ini.
6. Keluarga penulis (uti, akung, kakak, dan adik) yang selalu memberikan bantuan, dukungan, dan doa kepada penulis.
7. Sahabat penulis sejak bangku sekolah yang selalu memberikan semangat, dukungan, dan kebersamaan yang menguatkan penulis selama menjalani proses akademik.
8. Teman-teman Informatika UPN “Veteran” Yogyakarta angkatan 2022 yang tidak dapat penulis sebutkan satu per satu.

Semoga skripsi ini dapat memberikan manfaat bagi para pembaca. Penulis menyadari bahwa skripsi ini masih jauh dari kata sempurna karena keterbatasan ilmu pengetahuan yang dimiliki. Oleh karena itu, penulis sangat mengharapkan saran dan kritik yang membangun, serta pengembangan lebih lanjut agar penelitian ini dapat menjadi lebih baik di masa mendatang.

Yogyakarta, 19 April 2026

DAFTAR ISI

HALAMAN PENGESAHAN PEMBIMBING	iii
HALAMAN PENGESAHAN PENGUJI	iv
SURAT PERNYATAAN	v
KARYA ASLI TUGAS AKHIR	v
PERNYATAAN BEBAS PLAGIASI	vi
ABSTRAK	vii
ABSTRACT	viii
KATA PENGANTAR	ix
DAFTAR ISI	x
DAFTAR TABEL	xii
DAFTAR GAMBAR	xiii
DAFTAR CODE	xiv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah	2
1.4 Tujuan Penelitian	2
1.5 Manfaat Penelitian	3
1.6 Metodologi Penelitian dan Pengembangan Sistem	3
1.6.1 Metodologi Penelitian	3
1.6.2 Metodologi Pengembangan Sistem	4
1.7 Sistematika Penulisan	4
BAB II TINJAUAN LITERATUR	6
2.1 Konsep Monitoring Jaringan	6
2.2 Kesehatan Jaringan	6
2.2.1 Parameter Kesehatan Jaringan	6
2.2.2 Keterkaitan Antar Parameter Kesehatan Jaringan	8
2.3 Metode Deteksi Gangguan Jaringan	9
2.3.1 Pendekatan Rule-Based	9
2.3.2 Pendekatan Event Correlation	10
2.4 Lingkungan Simulasi Jaringan	10
2.5 MikroTik RouterOS dan Cloud Hosted Router(CHR)	11
2.6 Evaluasi Kinerja Sistem Monitoring	11
2.6.1 Mean Time to Detect (MTTD)	11
2.6.2 Detection Rate (DR)	12
2.7 Penelitian Terdahulu	12

BAB III METODOLOGI PENELITIAN	16
3.1 Metodologi Penelitian.....	16
3.1.1 Perancangan Lingkungan Uji	16
3.1.2 Pengumpulan dan Pengolahan Data	18
3.1.3 Perancangan Mekanisme Deteksi.....	23
3.1.4 Pengujian dan Evaluasi Kinerja Mekanisme Deteksi.....	26
3.2 Metodologi Pengembangan Sistem	29
3.2.1 Analisis Kebutuhan.....	30
3.2.2 Perancangan Desain.....	31
3.2.3 Implementasi	34
3.2.4 Pengujian.....	35
BAB IV HASIL PENGUJIAN DAN PEMBAHASAN	36
4.1 Implementasi	36
4.1.1 Implementasi Lingkungan Uji.....	36
4.1.2 Implementasi Pengambilan dan Pengolahan Data	40
4.1.3 Implementasi Rule-Based Detection.....	46
4.1.4 Implementasi Event Lifecycle	50
4.1.5 Implementasi Event Correlation.....	50
4.1.6 Implementasi Arsitektur Sistem.....	52
4.2 Hasil.....	54
4.2.1 Hasil Pengujian Mekanisme Deteksi.....	54
4.2.2 Hasil Pengujian Sistem.....	58
4.3 Pembahasan	58
BAB V PENUTUP	61
5.1 Kesimpulan.....	61
5.2 Saran.....	61
DAFTAR PUSTAKA	62

DAFTAR TABEL

Tabel 2.1 Kelas QoS Jaringan (International Telecommunication Union, 2011)	7
Tabel 2.2 Kelas QoS Jaringan (International Telecommunication Union, 2011) (Lanjutan)	8
Tabel 2.3 State of The Art	12
Tabel 2.4 State of The Art (Lanjutan)	13
Tabel 2.5 State of The Art (Lanjutan)	14
Tabel 2.6 State of The Art (Lanjutan)	15
Tabel 3.1 Spesifikasi Setiap Node Pada Topologi	17
Tabel 3.2 Baseline Kondisi Jaringan Uji	18
Tabel 3.3 Daftar Rule Pembentukan Event	23
Tabel 3.4 Daftar Pola Indikasi Jaringan	26
Tabel 3.5 Skenario Pengujian	27
Tabel 3.6 Skenario Pengujian (Lanjutan)	28
Tabel 3.7 Simulasi Perhitungan MTTD	29
Tabel 3.8 Spesifikasi Perangkat Keras	31
Tabel 3.9 Spesifikasi Perangkat Lunak	31
Tabel 3.10 Skenario Uji Sistem Monitoring	35
Tabel 4.1 Hasil Pengujian Mekanisme Deteksi	54
Tabel 4.2 Hasil Pengujian Mekanisme Deteksi (Lanjutan)	55
Tabel 4.3 Hasil Pengujian Mekanisme Deteksi (Lanjutan)	55
Tabel 4.4 Hasil Perhitungan Selisih Waktu Deteksi	56
Tabel 4.5 Hasil Pengujian Blackbox	58

DAFTAR GAMBAR

Gambar 3.1 Tahapan Penelitian	16
Gambar 3.2 Desain Topologi Lingkungan Uji.....	17
Gambar 3.3 Flowchart Mekanisme Pembentukan Event.....	24
Gambar 3.4 Flowchart Proses Event Correlation.....	26
Gambar 3.5 Metode Waterfall.....	30
Gambar 3.6 Arsitektur Sistem Monitoring.....	31
Gambar 3.7 Flowchart Sistem Monitoring.....	32
Gambar 3.8 Entity Relationship Diagram (ERD) Sistem Monitoring.....	33
Gambar 3.9 Wireframe Login Page Sistem Monitoring	33
Gambar 3.10 Wireframe Dashboard Sistem Monitoring	34
Gambar 4.1 Implementasi Topologi Lingkungan Uji	36
Gambar 4.2 Konektivitas Host ke Upstream Emulator dan MikroTik CHR	37
Gambar 4.3 Implementasi Network Emulator dengan Netem pada Upstream Emulator ..	38
Gambar 4.4 Konfigurasi IP dan Distribusi DHCP pada MikroTik CHR.....	39
Gambar 4.5 Konfigurasi Firewall Filter pada MikroTik CHR.....	39
Gambar 4.6 Konektivitas Antar Node Downstream dan Akses ke Jaringan Eksternal.....	40
Gambar 4.7 Implementasi Struktur Basis Data (Designer MySQL).....	52
Gambar 4.8 Tampilan Halaman Login.....	53
Gambar 4.9 Tampilan Dashboard Monitoring	53
Gambar 4.9 Tampilan Dashboard Monitoring (Lanjutan).....	54

DAFTAR CODE

Contoh Data 1. Respons endpoint GET /system/resource/.....	19
Contoh Data 2. Respons endpoint GET /interface/	19
Contoh Data 3. Respons endpoint POST /tool/ping.....	20
Contoh Data 4. Penyesuaian struktur data penggunaan CPU.....	21
Contoh Data 5. Penyesuaian struktur data penggunaan memori.....	21
Contoh Data 6. Penyesuaian struktur data delay jaringan.....	21
Contoh Data 7. Penyesuaian struktur data packet loss.....	21
Contoh Data 8. Penyesuaian struktur data status dan throughput interface.....	21
Contoh Data 9. Struktur sliding window penggunaan CPU.....	22
Contoh Data 10. Struktur sliding window penggunaan memori.....	22
Contoh Data 11. Struktur sliding window delay jaringan.....	22
Contoh Data 12. Struktur sliding window packet loss.....	22
Source Code 1. Mekanisme koneksi ke RouterOS API	40
Source Code 2. Pengambilan data identitas perangkat MikroTik.....	41
Source Code 3. Pengambilan data resource perangkat MikroTik.....	41
Source Code 4. Pengambilan data kualitas layanan jaringan.....	42
Source Code 5. Pengambilan data konektivitas interface.....	42
Source Code 6. Inisialisasi mekanisme poller.....	43
Source Code 7. Pengolahan data resource.....	43
Source Code 8. Pengolahan data kualitas layanan jaringan.....	44
Source Code 9. Pengolahan data konektivitas interface.....	44
Source Code 10. Pembentukan dan penambahan data pada sliding window.....	45
Source Code 11. Pengolahan Statistik Data pada Sliding Window.....	46
Source Code 12. Implementasi rule R1 (interface) kedalam rule engine.....	46
Source Code 13. Implementasi rule R2 (delay) kedalam rule engine.....	47
Source Code 14. Implementasi rule R3 (packet loss) kedalam rule engine.....	48
Source Code 15. Implementasi rule R4 (cpu) kedalam rule engine.....	49
Source Code 16. Implementasi rule engine pada poller.....	50
Source Code 17. Inisialisasi correlation engine.....	52

BAB I PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan teknologi informasi dan komunikasi telah meningkatkan ketergantungan berbagai sektor terhadap infrastruktur jaringan komputer yang stabil dan tersedia secara berkelanjutan. Dalam kondisi tersebut, monitoring jaringan menjadi komponen penting untuk menjaga kinerja perangkat, memastikan kualitas layanan, serta menjamin keberlangsungan operasional jaringan. Monitoring tidak hanya berfungsi sebagai sarana pengawasan, tetapi juga sebagai mekanisme untuk mendukung deteksi dini sebelum gangguan berdampak luas terhadap pengguna (Amin et al., 2025; Fachrurrozi et al., 2023).

Berdasarkan survei segmentasi pasar ISP tahun 2025, MikroTik merupakan merek router yang paling banyak digunakan oleh perusahaan ISP di Indonesia dengan tingkat penggunaan sebesar 53,00% (Asosiasi Penyelenggara Jasa Internet Indonesia, 2025). Tingginya tingkat adopsi tersebut menunjukkan bahwa peningkatan kualitas monitoring pada lingkungan MikroTik memiliki urgensi yang signifikan dalam konteks pengelolaan jaringan di Indonesia.

Seiring dengan tingginya penggunaan tersebut, MikroTik telah menyediakan berbagai fitur monitoring bawaan, seperti *The Dude* dan *Netwatch*, yang mampu memantau konektivitas perangkat jaringan (MikroTik, 2026b, 2026c). Fitur-fitur ini memungkinkan administrator untuk mengetahui perubahan kondisi perangkat secara langsung serta menerima notifikasi ketika terjadi gangguan. Namun, pemanfaatannya umumnya masih berfokus pada pemantauan parameter dan penyampaian perubahan kondisi, sehingga analisis terhadap hubungan antar kejadian masih bergantung pada interpretasi manual oleh administrator.

Sejumlah penelitian sebelumnya telah mengimplementasikan sistem monitoring jaringan berbasis MikroTik dengan pendekatan serupa, baik menggunakan fitur bawaan seperti *The Dude* dan *Netwatch* (Amin et al., 2025; Irawansyah et al., 2024; Maestro Pasaribu & Ferdiansyah, 2024; Pratama et al., 2025; Rahayu & Prisma, 2022) maupun melalui integrasi dengan sistem monitoring enterprise seperti Zabbix dan LibreNMS (Caicedo-Altamirano, Villacrés-Cevallos, et al., 2025; Fachrurrozi et al., 2023; Malik & Josaphat, 2024; Pillo-Guanoluisa et al., 2025). Penelitian-penelitian tersebut umumnya mampu menyajikan informasi jaringan secara real-time, namun sebagian besar informasi kesehatan jaringan yang disampaikan masih berorientasi pada kejadian tunggal.

Pada praktiknya, gangguan jaringan jarang muncul sebagai gejala tunggal, melainkan merupakan kombinasi beberapa gejala yang saling berkaitan (Troia et al., 2023). Sebagai contoh, penurunan kualitas layanan dapat dipengaruhi oleh berbagai faktor seperti kondisi trafik, beban perangkat, dan faktor eksternal lainnya (Troia et al., 2023). Oleh karena itu, analisis yang hanya berfokus pada satu kejadian tidak cukup untuk menggambarkan kondisi gangguan secara menyeluruh. Diperlukan pendekatan yang mampu mengolah keterkaitan antar kejadian agar menghasilkan informasi yang lebih kontekstual dan bermakna.

Berdasarkan permasalahan tersebut, penelitian ini mengusulkan pengembangan sistem monitoring jaringan berbasis MikroTik dengan penerapan *rule-based detection* dan *event correlation*. *Rule-based detection* digunakan untuk membentuk event gangguan berdasarkan kondisi parameter kesehatan jaringan, sedangkan *event correlation* digunakan untuk mengaitkan event dalam rentang waktu tertentu guna menghasilkan indikasi awal gangguan jaringan. Pendekatan ini diharapkan mampu menghasilkan informasi yang lebih kontekstual serta membantu administrator memahami kemungkinan sumber gangguan secara lebih terarah dengan tetap menjaga keterlacakan proses analisis.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, masalah utama yang akan dibahas dalam penelitian ini adalah:

1. Kebutuhan akan monitoring jaringan berbasis MikroTik yang mampu mengolah keterkaitan antar event untuk menghasilkan indikasi awal gangguan.
2. Penyajian informasi monitoring jaringan berbasis MikroTik masih berorientasi pada kejadian tunggal yang kurang kontekstual.
3. Analisis hubungan antar kejadian masih bergantung pada interpretasi manual sehingga identifikasi indikasi awal gangguan kurang efisien.
4. Belum diketahuinya efektivitas pendekatan *rule-based* dan *event correlation* dalam mendeteksi indikasi awal gangguan jaringan.

1.3 Batasan Masalah

Agar penelitian ini lebih terarah, batasan masalah ditetapkan sebagai berikut:

1. Sistem monitoring dibatasi hanya untuk perangkat jaringan MikroTik.
2. Data analisis dibatasi pada beberapa parameter kesehatan jaringan yang diperoleh dari perangkat MikroTik.
3. Konfigurasi *rule-based detection* dan pola *event correlation* ditetapkan berdasarkan literatur serta kondisi baseline eksperimen.
4. Implementasi dan pengujian dilakukan pada lingkungan simulasi menggunakan GNS3 dengan MikroTik CHR sebagai representasi router.
5. Evaluasi kinerja system monitoring dibatasi pada *Detection Rate* (DR) dan *Mean Time To Detect* (MTTD) berdasarkan hasil pengujian.

1.4 Tujuan Penelitian

Berdasarkan perumusan masalah yang ada, tujuan dari penelitian ini adalah:

1. Terbentuknya sistem monitoring jaringan berbasis MikroTik yang mampu mengolah keterkaitan antar event untuk menghasilkan indikasi awal gangguan.
2. Terbentuknya mekanisme *rule-based detection* untuk membentuk event berdasarkan parameter kesehatan jaringan secara lebih kontekstual.
3. Terbentuknya mekanisme *event correlation* untuk menghasilkan indikasi awal gangguan jaringan berdasarkan keterkaitan antar event.
4. Diketahuinya gambaran efektivitas pendekatan *rule-based detection* dan *event correlation* dalam mendeteksi indikasi awal gangguan jaringan.

1.5 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat sebagai berikut:

1. Manfaat Teoretis

Penelitian ini memberikan kontribusi dalam penerapan pendekatan *rule-based* dan *event correlation* pada sistem monitoring jaringan berbasis MikroTik untuk penyusunan indikasi awal gangguan. Pendekatan yang digunakan dapat menjadi rujukan dalam penelitian selanjutnya dengan konteks dan lingkup yang serupa.

2. Manfaat Praktis

Penelitian ini menghasilkan sistem monitoring berbasis MikroTik yang tidak hanya menampilkan kondisi jaringan, tetapi juga menyusun indikasi awal gangguan secara terstruktur. Sistem ini diharapkan dapat membantu administrator jaringan dalam memahami kondisi gangguan secara lebih cepat dan terarah sebagai dasar pengambilan keputusan.

1.6 Metodologi Penelitian dan Pengembangan Sistem

Alur metode penelitian dan pengembangan sistem pada penelitian ini dilakukan sebagai berikut:

1.6.1 Metodologi Penelitian

Tahapan dari metodologi penelitian yang dilakukan adalah sebagai berikut:

1. Pengumpulan dan Pengolahan Parameter Kesehatan Jaringan

Tahap ini dilakukan pengambilan data dari perangkat MikroTik melalui modul pengambilan data. Data yang diambil kemudian diolah dengan menyamakan format, satuan, dan kemudian dimasukkan kedalam sliding window untuk ditampilkan pada dashboard dan dianalisis lebih lanjut.

2. Desain Lingkungan Uji

Tahap ini dilakukan perancangan topologi jaringan sebagai simulasi lingkungan uji menggunakan GNS3 serta MikroTik CHR dan menetapkan baseline kondisi jaringan uji.

3. Perancangan Mekanisme Analisis Gangguan Berbasis Event

Tahap ini dilakukan perancangan mekanisme *rule-based detection* untuk membentuk event berdasarkan threshold dan durasi pelanggaran threshold. Kemudian dilakukan perancangan mekanisme siklus hidup event (*event lifecycle*). Serta, perancangan *event correlation* untuk menganalisis indikasi gangguan jaringan berbasis waktu kejadian event dan keterkaitan antar parameter pembentuk event.

4. Pengujian dan Evaluasi Kinerja Mekanisme Analisis Gangguan

Tahap akhir dilakukan pengujian kemampuan mekanisme deteksi dengan skenario gangguan yang telah dirancang. Kemudian dilakukan perhitungan akurasi dan kecepatan deteksi berdasarkan hasil pengujian.

1.6.2 Metodologi Pengembangan Sistem

Metode pengembangan sistem yang digunakan dalam penelitian ini yaitu *Waterfall Model*. Pendekatan ini dipilih karena sesuai dengan kebutuhan penelitian ini yang sifatnya terstruktur tidak berubah-ubah. Tahapan yang dilakukan adalah sebagai berikut:

1. Analisis Kebutuhan
Tahap ini dilakukan identifikasi kebutuhan fungsional maupun non fungsional dari sistem monitoring yang akan dikembangkan dalam penelitian ini.
2. Perancangan Desain
Tahap ini dilakukan perancangan desain sistem monitoring yang akan dikembangkan, mulai dari desain arsitektur, alur sistem, desain database, hingga wireframe antarmuka sistem.
3. Implementasi
Tahap ini yaitu tahap pengembangan sistem sesuai dengan kebutuhan dan desain yang sudah di rancang sebelumnya.
4. Pengujian
Tahap ini dilakukan pengujian fungsionalitas sistem monitoring menggunakan metode blackbox testing.

1.7 Sistematika Penulisan

Bagian ini berisi struktur Tugas Akhir ini mulai Bab Pendahuluan sampai Bab Penutup dan deskripsi singkat dari masing-masing bab. Sistematika penulisan laporan pada penelitian ini adalah sebagai berikut :

1. BAB I PENDAHULUAN

Bab ini membahas latar belakang permasalahan yang melandasi penelitian, rumusan masalah, batasan masalah, tujuan dan manfaat penelitian, tahapan penelitian yang digunakan, serta sistematika penulisan laporan. Bab ini berfungsi untuk memberikan konteks penelitian serta menjelaskan alasan pemilihan pendekatan monitoring kesehatan jaringan berbasis MikroTik dengan metode *rule-based* dan *event correlation*.

2. BAB II TINJAUAN LITERATUR

Bab ini membahas landasan teoretis terkait monitoring jaringan, parameter kesehatan jaringan, metode analisis penyebab gangguan, sistem *rule-based*, dan *event correlation*. Pada bab ini juga ditegaskan ruang lingkup penelitian dalam kerangka teori melalui pemilihan parameter dan pendekatan analisis yang digunakan, serta dilakukan pemetaan penelitian terdahulu untuk mengidentifikasi *research gap* dan posisi kontribusi penelitian.

3. BAB III METODOLOGI PENELITIAN

Bab ini membahas tahapan penelitian menggunakan model pengembangan Agil, perancangan sistem monitoring, lingkungan simulasi jaringan, proses akuisisi dan *pre-processing* data, mekanisme *rule-based detection*, pembentukan serta siklus hidup event, metode *event correlation*, serta rancangan pengujian dan evaluasi kinerja sistem.

4. BAB IV HASIL PENGUJIAN DAN PEMBAHASAN

Bab ini membahas implementasi *system* berdasarkan rancangan, hasil pengujian pada berbagai skenario, serta analisis kinerja sistem dalam mendeteksi kondisi jaringan dan menyusun indikasi awal penyebab gangguan.

5. BAB V PENUTUP

Bab ini membahas kesimpulan yang merangkum hasil penelitian dan menjawab rumusan masalah, serta saran untuk pengembangan penelitian selanjutnya.

BAB II

TINJAUAN LITERATUR

2.1 Konsep Monitoring Jaringan

Monitoring jaringan merupakan proses pengawasan kondisi jaringan komputer secara berkelanjutan untuk menjaga kinerja perangkat, memastikan kualitas layanan, serta menjamin keberlangsungan operasional jaringan. Monitoring dilakukan dengan memantau parameter jaringan, seperti lalu lintas data dan status perangkat, untuk memperoleh informasi kondisi jaringan (Amin et al., 2025).

Informasi tersebut digunakan untuk mendeteksi gangguan dan perubahan kondisi jaringan, serta mendukung administrator dalam menjaga stabilitas dan mengambil keputusan secara tepat. Sistem monitoring yang efektif mampu menyajikan informasi secara *real-time* dan memberikan notifikasi ketika terjadi gangguan atau perubahan status perangkat (Amin et al., 2025; Fachrurrozi et al., 2023).

2.2 Kesehatan Jaringan

Kesehatan jaringan (*network health*) merupakan kondisi operasional jaringan yang menunjukkan apakah jaringan mampu menjalankan layanan sesuai kebutuhan berdasarkan parameter performa yang terukur. Dalam monitoring jaringan, kondisi ini dinilai berdasarkan metrik seperti delay, packet loss, dan konektivitas yang merepresentasikan kualitas layanan dan kondisi jaringan secara keseluruhan (Alkenani & Nassar, 2022).

Jaringan dikategorikan sehat apabila nilai parameter tersebut berada dalam batas normal sehingga trafik dapat diproses tanpa gangguan yang signifikan. Sebaliknya, jaringan dikategorikan tidak sehat ketika terjadi penurunan performa, seperti peningkatan delay atau bertambahnya packet loss yang menunjukkan adanya masalah pada proses pengiriman data (Alkenani & Nassar, 2022).

Selain parameter kualitas layanan, kondisi sumber daya juga memengaruhi kesehatan jaringan. Beban CPU yang tinggi hingga mencapai titik jenuh dapat menjadi bottleneck dan menyebabkan penurunan performa, yang berdampak pada meningkatnya delay dan packet loss (Troia et al., 2023).

2.2.1 Parameter Kesehatan Jaringan

Implementasi monitoring jaringan memerlukan parameter yang mampu merepresentasikan kondisi operasional jaringan secara terukur. Parameter ini digunakan sebagai indikator untuk menilai apakah jaringan berada dalam kondisi normal atau mengalami gangguan. Monitoring jaringan melibatkan pengukuran berbagai metrik performa untuk mengevaluasi kondisi jaringan, seperti penggunaan sumber daya perangkat (CPU, memori), kualitas layanan jaringan (*Quality of Service/QoS*) yang mencakup delay, jitter, packet loss, dan throughput, serta status konektivitas perangkat (*up/down*) (Alkenani & Nassar, 2022; Caicedo-Altamirano, Villacrés-Cevallos, et al., 2025; Fachrurrozi et al., 2023).

1. Sumber Daya Perangkat

Parameter performa perangkat digunakan untuk merepresentasikan kondisi sumber daya internal router yang berperan dalam pemrosesan paket dan penyediaan layanan jaringan. Secara umum, performa perangkat jaringan dapat diamati melalui beberapa indikator, seperti penggunaan CPU dan memori perangkat.

Penggunaan CPU (CPU utilization) menunjukkan tingkat beban prosesor dalam menjalankan proses routing dan layanan sistem. Nilai CPU yang tinggi dan berlangsung secara terus-menerus dapat mengindikasikan kondisi beban berlebih yang berpotensi menurunkan performa dan kualitas layanan jaringan (Troia et al., 2023). Ambang penggunaan CPU umumnya ditetapkan pada kisaran 80%-90% sebagai indikator beban tinggi perangkat dan sering digunakan sebagai nilai trigger pada template monitoring seperti Zabbix (Caicedo-Altamirano, Villacrés-Cevallos, et al., 2025).

Penggunaan memori menunjukkan kemampuan perangkat dalam menangani proses dan data sementara. Nilai memori yang tinggi dapat menyebabkan penurunan performa dan mengganggu stabilitas layanan jaringan.

2. Kualitas Layanan Jaringan (Quality of Service/QoS)

Quality of Service (QoS) merupakan ukuran performa jaringan yang digunakan untuk menilai kemampuan jaringan dalam memenuhi kebutuhan layanan berdasarkan parameter terukur. Untuk memastikan pengukuran yang terstandarisasi, ITU-T menetapkan model evaluasi QoS pada jaringan IP melalui rekomendasi Y.1540 dan Y.1541. Rekomendasi Y.1540 mendefinisikan parameter performa jaringan, sedangkan Y.1541 mengelompokkan kualitas layanan ke dalam beberapa kelas beserta target performanya sebagai acuan evaluasi (International Telecommunication Union, 2011, 2019).

Parameter QoS yang digunakan dalam standar tersebut meliputi delay (IP Packet Transfer Delay/IPTD), variasi delay (IP Packet Delay Variation/IPDV), serta packet loss dan error (IP Packet Loss Ratio/IPLR dan IP Packet Error Ratio/IPER). Parameter-parameter ini menjadi indikator utama dalam menilai kualitas layanan jaringan, khususnya pada jaringan berbasis paket (Alkenani & Nassar, 2022; International Telecommunication Union, 2019). Klasifikasi QoS berdasarkan ITU-T Y.1541 beserta contoh peruntukan layanan ditunjukkan pada Tabel 2.1.

Tabel 2.1 Kelas QoS Jaringan (International Telecommunication Union, 2011)

Class	IPTD	IPDV	IPLR	IPER	Kategori Penggunaan
Class 0	100 ms	50 ms	0.1 %	0.01%	Aplikasi real-time sangat sensitif (VoIP, video conference)
Class 1	400 ms	50 ms	0.1 %	0.01%	Aplikasi real-time dengan toleransi delay lebih tinggi
Class 2	100 ms	U	0.1 %	0.01%	Transaksi interaktif (signaling, kontrol jaringan)
Class 3	400 ms	U	0.1 %	0.01%	Transaksi interaktif dengan toleransi delay lebih longgar

Tabel 2.2 Kelas QoS Jaringan (International Telecommunication Union, 2011)
(Lanjutan)

Class	IPTD	IPDV	IPLR	IPER	Kategori Penggunaan
Class 4	1000 ms	U	0.1 %	0.01%	Aplikasi non-real-time (streaming, transfer data)
Class 5	U	U	U	U	Layanan best-effort tanpa jaminan QoS
Keterangan: U (<i>Unspecified</i>) menunjukkan bahwa parameter tersebut tidak diberikan batas nilai tertentu dalam kelas layanan yang bersangkutan					

Berdasarkan klasifikasi tersebut, setiap jenis layanan memiliki kebutuhan QoS yang berbeda sehingga tidak terdapat satu ambang batas yang berlaku universal. Oleh karena itu, dalam konteks monitoring jaringan, evaluasi QoS dilakukan dengan membandingkan hasil pengukuran terhadap target performa pada kelas layanan yang relevan.

Analisis QoS pada penelitian ini difokuskan pada parameter delay dan packet loss karena keduanya telah cukup merepresentasikan kualitas jalur komunikasi. Pada standar ITU-T, delay didefinisikan sebagai one-way delay, yaitu waktu tempuh paket dari sumber ke tujuan. Namun, penelitian ini menggunakan Round-Trip Time (RTT), yaitu waktu tempuh paket pulang-pergi, menyesuaikan dengan metode pengukuran pada perangkat Mikrotik. Dalam kondisi jaringan yang simetris, RTT dapat dianggap sebagai dua kali one-way delay (Alkenani & Nassar, 2022; International Telecommunication Union, 2019).

3. Konektivitas Perangkat

Konektivitas perangkat merujuk pada ketersediaan jalur komunikasi pada interface jaringan yang menghubungkan antar perangkat dalam suatu jaringan. Konektivitas ini menunjukkan apakah komunikasi data masih dapat berlangsung atau tidak.

Indikator utama dalam menilai konektivitas adalah status interface (up/down). Status up menunjukkan bahwa interface aktif dan jalur komunikasi tersedia, sedangkan status down menunjukkan bahwa koneksi terputus sehingga komunikasi tidak dapat dilakukan.

Dalam sistem monitoring jaringan, perubahan status interface ini umum dimanfaatkan untuk mendeteksi gangguan secara real-time. Sistem monitoring akan memberikan notifikasi ketika terjadi perubahan kondisi, seperti dari up menjadi down atau sebaliknya, sehingga administrator dapat segera mengetahui adanya masalah pada jaringan (Maestro Pasaribu & Ferdiansyah, 2024).

2.2.2 Keterkaitan Antar Parameter Kesehatan Jaringan

Parameter QoS seperti delay dan packet loss mencerminkan kualitas komunikasi end-to-end dalam jaringan. Peningkatan kedua parameter ini umumnya menunjukkan adanya gangguan pada jalur komunikasi, misalnya akibat kepadatan trafik atau masalah jaringan pada sisi upstream seperti ISP (Alkenani & Nassar, 2022).

Namun, penurunan kualitas jaringan tidak selalu disebabkan oleh faktor eksternal. Keterbatasan sumber daya perangkat, seperti tingginya penggunaan CPU pada router, juga dapat memengaruhi kemampuan perangkat dalam memproses paket dan berdampak pada meningkatnya delay serta packet loss (Troia et al., 2023).

Oleh karena itu, hubungan antar parameter dapat digunakan untuk membantu mengidentifikasi sumber gangguan. Jika delay dan packet loss meningkat sementara penggunaan CPU normal, maka gangguan cenderung berasal dari jaringan, seperti pada jalur komunikasi atau koneksi ke ISP (Alkenani & Nassar, 2022). Sebaliknya, jika peningkatan tersebut disertai dengan penggunaan CPU yang tinggi, maka gangguan lebih mengarah pada keterbatasan perangkat (Troia et al., 2023).

Di sisi lain, status interface menunjukkan kondisi konektivitas secara langsung (up/down), sehingga dapat digunakan sebagai indikator awal dalam mendeteksi terjadinya gangguan jaringan (Maestro Pasaribu & Ferdiansyah, 2024).

2.3 Metode Deteksi Gangguan Jaringan

Deteksi gangguan jaringan dalam konteks monitoring bertujuan menafsirkan gejala yang teramati menjadi indikasi awal penyebab gangguan. Gejala tersebut dapat berupa perubahan parameter performa, degradasi kualitas layanan, maupun terputusnya konektivitas. Untuk menghindari interpretasi yang parsial terhadap setiap gejala, diperlukan pendekatan analitis yang mampu menghubungkan kondisi-kondisi tersebut secara sistematis. Bagian ini membahas dua pendekatan yang relevan dalam analisis gangguan jaringan, yaitu *rule-based* dan *event correlation*, sebagai landasan teoretis penelitian.

2.3.1 Pendekatan Rule-Based

Pendekatan *rule-based* merupakan metode berbasis pengetahuan yang merepresentasikan keahlian pakar dalam bentuk aturan logis *if-then* untuk mendukung pengambilan keputusan secara konsisten. Sistem ini termasuk dalam kategori *rule-based expert system* yang terdiri atas *knowledge base* yang menyimpan fakta dan aturan, serta *inference engine* yang bertugas mengevaluasi aturan tersebut untuk menghasilkan kesimpulan atau rekomendasi (De Gasperis & Facchini, 2025; Kodors et al., 2025). Proses inferensi dilakukan dengan mencocokkan kondisi pada bagian *if* suatu aturan dan mengeksekusi bagian *then* ketika kondisi tersebut terpenuhi.

Pendekatan *rule-based* memiliki keunggulan dalam mendeteksi insiden yang telah terdefinisi dengan pola yang jelas, karena kondisi tersebut dapat dipetakan secara langsung ke dalam aturan eksplisit (De Gasperis & Facchini, 2025). Selain itu, tingkat interpretabilitas yang tinggi memungkinkan setiap event atau keputusan yang dihasilkan ditelusuri kembali ke aturan yang aktif. Karakteristik ini menjadikan pendekatan *rule-based* mudah dipahami dan diaudit, sehingga sesuai untuk diterapkan pada lingkungan operasional yang menuntut transparansi pengambilan keputusan.

Namun, karena bergantung pada aturan yang telah ditentukan sebelumnya, pendekatan ini kurang fleksibel dalam menangani insiden yang melibatkan interaksi parameter yang beragam atau pola yang belum terdefinisi.

Dengan mempertimbangkan karakteristik tersebut, *rule-based* dalam penelitian ini diposisikan sebagai tahap awal pembentukan event yang terdefinisi dan dapat dijelaskan. Event yang dihasilkan selanjutnya dianalisis keterkaitannya melalui mekanisme event *correlation* untuk menyusun indikasi awal gangguan jaringan berdasarkan pola kemunculan dan kombinasi event.

2.3.2 Pendekatan Event Correlation

Event correlation merupakan proses untuk mengidentifikasi dan menghubungkan beberapa event yang memiliki keterkaitan tertentu sehingga event-event tersebut tidak diperlakukan sebagai kejadian yang berdiri sendiri. Melalui proses ini, hubungan antar event dapat dianalisis secara bersama untuk menghasilkan informasi yang lebih bermakna dalam konteks monitoring sistem (Kotenko et al., 2022).

Kotenko et al. (2022) mengelompokkan metode *event correlation* ke dalam beberapa pendekatan, antara lain *similarity-based*, *step-based (sequential)*, *causal*, *statistical/data-driven*, serta pendekatan *hybrid* yang menggabungkan beberapa metode. Pendekatan *similarity-based* mengaitkan event berdasarkan kesamaan atribut tertentu, seperti waktu kemunculan atau karakteristik parameter, sedangkan *step-based* menghubungkan event berdasarkan urutan kejadian. Pendekatan *causal* memodelkan hubungan sebab-akibat antar event, dan pendekatan *statistical* memanfaatkan pola yang diperoleh dari data historis.

Berdasarkan klasifikasi tersebut, penelitian ini menerapkan *temporal similarity-based event correlation*, yaitu mengaitkan event berdasarkan waktu kemunculannya. Pendekatan ini dipilih karena gangguan jaringan umumnya menyebabkan beberapa parameter berubah dalam waktu yang berdekatan, sehingga kedekatan temporal sudah memadai untuk mengelompokkan event sebagai satu indikasi awal kondisi sistem tanpa memodelkan hubungan sebab-akibat yang kompleks. Hasil proses korelasi berupa indikasi awal gangguan berdasarkan keterkaitan event-event yang muncul, yang selanjutnya dapat menjadi dasar analisis bagi administrator jaringan.

2.4 Lingkungan Simulasi Jaringan

Lingkungan simulasi jaringan digunakan dalam penelitian sistem monitoring untuk menyediakan kondisi eksperimen yang terkontrol dan dapat direplikasi sebelum diterapkan pada jaringan produksi (Gomez et al., 2023). Platform eksperimen jaringan umumnya diklasifikasikan menjadi *simulator*, *emulator*, dan *testbed*, yang masing-masing menawarkan tingkat kontrol, realisme, dan kebutuhan sumber daya yang berbeda.

Simulator dan *emulator* memungkinkan pengujian yang fleksibel dan efisien dari sisi biaya, sedangkan *testbed* berbasis perangkat fisik memberikan tingkat realisme lebih tinggi dengan konsekuensi kebutuhan infrastruktur yang lebih besar (Gomez et al., 2023). Dalam konteks penelitian jaringan, laboratorium virtual berbasis *emulator* seperti GNS3 telah digunakan untuk merepresentasikan topologi dan protokol jaringan secara memadai pada tahap evaluasi awal (Caicedo-Altamirano, Alpúsig-Cuichán, et al., 2025). Dengan demikian, pendekatan simulasi dan emulasi dipandang relevan sebagai lingkungan pengujian awal sistem monitoring sebelum diterapkan pada jaringan operasional.

2.5 MikroTik RouterOS dan Cloud Hosted Router(CHR)

MikroTik RouterOS merupakan sistem operasi jaringan yang digunakan pada perangkat router MikroTik, baik dalam bentuk perangkat fisik maupun virtual. RouterOS berbasis kernel Linux dan menyediakan berbagai fungsi jaringan seperti *routing IP*, *firewall*, manajemen *bandwidth*, VPN, serta fasilitas monitoring melalui antarmuka baris perintah, aplikasi grafis, dan *Application Programming Interface* (API) (MikroTik, 2026d). Ketersediaan API memungkinkan integrasi dengan sistem eksternal untuk kebutuhan pengambilan data operasional dan pengembangan sistem monitoring.

Cloud Hosted Router (CHR) adalah distribusi RouterOS yang berjalan pada lingkungan virtual berbasis arsitektur *x86-64*. CHR mempertahankan fitur inti RouterOS dengan perbedaan utama pada aspek lisensi dan tidak bergantung pada perangkat keras khusus (MikroTik, 2026a). Dalam konteks penelitian jaringan, CHR banyak dimanfaatkan sebagai representasi virtual router karena fleksibel dan mendukung eksperimen pada lingkungan laboratorium. Tingginya tingkat adopsi MikroTik pada ISP di Indonesia (APJII, 2025) menjadikan RouterOS relevan sebagai objek kajian dalam penelitian monitoring jaringan.

2.6 Evaluasi Kinerja Sistem Monitoring

Evaluasi kinerja sistem monitoring kesehatan jaringan dilakukan untuk menilai kemampuan sistem dalam mendeteksi indikasi gangguan dan kondisi jaringan berdasarkan informasi yang diperoleh dari perangkat jaringan.

Sistem yang dikembangkan menggunakan pendekatan *rule-based detection* dan *event correlation* untuk mengidentifikasi indikasi gangguan jaringan. Oleh karena itu, evaluasi difokuskan pada kecepatan serta tingkat keberhasilan sistem dalam mendeteksi gangguan.

Pendekatan evaluasi mengadaptasi metrik yang umum digunakan pada sistem deteksi berbasis aturan, seperti *Intrusion Detection System* (IDS), karena memiliki kesamaan dalam mekanisme deteksi berbasis event. Metrik yang digunakan meliputi *Mean Time to Detect* (MTTD) dan *Detection Rate*.

2.6.1 Mean Time to Detect (MTTD)

Mean Time to Detect (MTTD) merupakan metrik yang digunakan untuk mengukur rata-rata waktu yang dibutuhkan sistem dalam mendeteksi gangguan sejak pertama kali terjadi (ElSahly & Abdelfatah, 2022; Syairozi & Arizal, 2025). Nilai MTTD dihitung sebagai rata-rata selisih antara waktu terjadinya gangguan dan waktu deteksi oleh sistem (ElSahly & Abdelfatah, 2022), sebagaimana ditunjukkan pada Persamaan (2.1).

$$MTTD = \frac{\sum_{i=1}^{Ndet} (Tdet_i - Tins_i)}{Ndet} \dots\dots\dots (2.1)$$

Keterangan:

$Tdet_i$: waktu insiden ke-i terdeteksi oleh sistem

$Tins_i$: waktu insiden ke-i mulai terjadi

N : jumlah insiden yang terdeteksi oleh sistem

2.6.2 Detection Rate (DR)

Detection Rate (DR) digunakan untuk mengukur tingkat keberhasilan sistem dalam mendeteksi gangguan secara benar atau valid (ElSahly & Abdelfatah, 2022, Syairozi & Arizal, 2025). Nilai DR dihitung sebagai persentase jumlah kejadian yang berhasil terdeteksi dengan benar dibandingkan dengan total kejadian yang diuji (ElSahly & Abdelfatah, 2022), sebagaimana ditunjukkan pada Persamaan (2.2).

$$DR = \frac{\text{Total insiden terdeteksi benar}}{\text{Total insiden yang diujikan}} \times 100\% \dots\dots\dots (2.2)$$

2.7 Penelitian Terdahulu

Untuk memastikan posisi dan kontribusi penelitian ini, Tabel 2.3 merangkum dan membandingkan penelitian-penelitian terdahulu yang relevan pada topik monitoring jaringan berbasis MikroTik.

Tabel 2.3 State of The Art

No.	Penelitian	Metode	Hasil Penelitian
1.	Judul: Cloud-Based Network Monitoring System with Zabbix and WireGuard VPN for Remote SMEs Penulis: Fernando Caicedo-Altamirano, Patricio Villacrés-Cevallos, Alex Ramos-Jinez, Silvia Alpusig-Cuichan, Omar Guerra-Pazo Tahun: 2025	Mengimplementasikan sistem monitoring jaringan berbasis cloud menggunakan Zabbix yang terintegrasi dengan WireGuard VPN pada AWS EC2. Metode meliputi deployment server monitoring di cloud, konfigurasi VPN tunnel untuk koneksi aman, serta pemanfaatan SNMP dan trigger berbasis threshold untuk pemantauan parameter jaringan.	Sistem mampu melakukan monitoring real-time pada jaringan remote secara aman melalui VPN dengan pemantauan parameter seperti CPU, traffic, dan storage, serta menghasilkan notifikasi indikasi gangguan seperti kegagalan interface, overload CPU, dan saturasi resource yang dipicu oleh kondisi threshold pada parameter jaringan (Caicedo-Altamirano, Villacrés-Cevallos, et al., 2025).

Tabel 2.4 State of The Art (Lanjutan)

No.	Penelitian	Metode	Hasil Penelitian
2.	Judul: Real-Time Network Infrastructure Monitoring A Zabbix and Telegram Integration Case Study in the Energy Sector (EMELNORTE S.A.) Penulis: Darwin Pillo-Guanoluisa, Leandro Bermúdez Herrera, Alexandra Juma Alba, Diego Salgado Báez, Victoria Fray Buestán Tahun: 2025	Mengimplementasikan sistem monitoring terpusat menggunakan Zabbix berbasis SNMP dan ICMP pada VMware ESXi dengan notifikasi Telegram dan email, menggunakan metode PPDIOO dan framework FCAPS untuk identifikasi metrik (availability, traffic, interface, CPU, memory).	Sistem mampu melakukan monitoring real-time dan mendeteksi anomali seperti lonjakan traffic dan utilisasi interface, serta mengirim notifikasi gangguan secara cepat dengan informasi detail (severity, host, durasi, dan identifikasi masalah), sehingga waktu respons insiden menurun signifikan. (Pillo-Guanoluisa et al., 2025)
3.	Judul: Design of Network Monitoring System Based on LibreNMS Using Line Notify, Telegram, and Email Notification Penulis: Nurwan Reza Fachrurrozi, Andri Agustav Wirabudi, Seandy Arandiant Rozano Tahun: 2023	Mengembangkan sistem monitoring jaringan terpusat berbasis LibreNMS dengan integrasi notifikasi melalui Line Notify, Telegram, dan Email untuk pemantauan parameter seperti CPU, memory, traffic, bandwidth, dan status perangkat. Menggunakan metode NDLC dan PPDIOO	Sistem mampu memonitor perangkat jaringan secara menyeluruh dan menampilkan data real-time (resource, traffic, log), serta mengirim notifikasi otomatis saat terjadi perubahan kondisi (up/down, reboot), sehingga meningkatkan ketersediaan dan keandalan jaringan (Fachrurrozi et al., 2023).
4.	Judul: Designing a Computer Network Monitoring System With SMS Notification Using The Dude Penulis: Riki Irawansyah, Khairil, Rizka Tri Alinse Tahun: 2024	Merancang sistem monitoring jaringan untuk mengetahui kondisi perangkat secara jarak jauh, dengan memanfaatkan The Dude sebagai tools monitoring berbasis MikroTik yang bekerja berdasarkan IP address, serta mengintegrasikan notifikasi SMS untuk mendeteksi perubahan status perangkat (up/down).	Sistem mampu menampilkan kondisi perangkat secara visual (up/down) dan mengirimkan notifikasi SMS kepada administrator, sehingga mempermudah deteksi gangguan jaringan secara cepat dan remote (Irawansyah et al., 2024).
5.	Judul: Implementation of Network and Server Monitoring on MikroTik with Netwatch Via Telegram Bot Penulis: Yohanes Maestro Pasaribu, Ferdiansyah Tahun: 2024	Mengimplementasikan sistem monitoring jaringan dan server pada MikroTik menggunakan fitur Netwatch dengan integrasi Telegram Bot. Menggunakan metode pengembangan ADDIE (Analysis, Design, Development, Implementation, Evaluation) serta konfigurasi script pada MikroTik untuk pengiriman log dan request monitoring.	Sistem mampu melakukan monitoring jaringan dan server secara real-time melalui Telegram Bot, termasuk status koneksi (up/down), penggunaan resource, serta kontrol perangkat jaringan. Notifikasi dikirim otomatis dan berbasis request sehingga memudahkan administrator dalam pemantauan dan pengendalian jaringan secara jarak jauh (Maestro Pasaribu & Ferdiansyah, 2024).

Tabel 2.5 State of The Art (Lanjutan)

No.	Penelitian	Metode	Hasil Penelitian
6.	Judul: Design and Implementation of the Dude MikroTik Server Using Telegram Bot as Monitoring Client Network and Internet Connection Penulis: Ahmad Bagus Putra Pratama, Eko Handoyo, Muhammad Shodiq Tahun: 2025	Mengimplementasikan sistem monitoring jaringan menggunakan The Dude pada Mikrotik dengan integrasi Telegram Bot. Metode yang digunakan adalah NDLC (Analysis, Design, Simulation, Implementation, Monitoring) serta simulasi menggunakan Cisco Packet Tracer untuk perancangan topologi.	Sistem mampu melakukan monitoring jaringan secara real-time dengan pemetaan perangkat dan deteksi status (up/down), serta mengirim notifikasi otomatis ke Telegram saat terjadi gangguan jaringan, seperti server down serta indikasi masalah bandwidth atau konektivitas perangkat dan saat koneksi kembali normal. Sehingga mengurangi downtime dan meningkatkan respons terhadap gangguan jaringan (Pratama et al., 2025).
7.	Judul: Implementasi Monitoring Jaringan Internet pada Mikrotik Menggunakan Notifikasi Bot Telegram di SMK Penulis: Munawar Amin, Muhammad Hasti, Bebas Widada, Bramasto Wiryawan Yudanto Tahun: 2025	Mengimplementasikan sistem monitoring jaringan berbasis Mikrotik menggunakan fitur Netwatch dengan integrasi Bot Telegram. Metode yang digunakan adalah NDLC (Analysis, Design, Simulation, Implementation, Monitoring, Management) serta simulasi menggunakan GNS3 dan konfigurasi melalui Winbox	Sistem mampu mendeteksi perubahan status perangkat jaringan secara real-time dan mengirimkan notifikasi otomatis ke Telegram, seperti perangkat disconnect, aktivitas login/logout, serta gangguan koneksi VPN (PPPoE/PPTP). Notifikasi memuat informasi perangkat, lokasi, dan status (up/down) untuk mempercepat penanganan gangguan dan mengurangi downtime (Amin et al., 2025).
8.	Judul: Sistem Monitoring Jaringan Berbasis Web Dengan Fitur Notifikasi Insiden Otomatis dan Pencatatan Log Historis pada Pt. Mitra Bersama Jaya Penulis: Wildi Baydowi, Hidayatullah Al Islami Tahun: 2026	Mengembangkan sistem monitoring jaringan berbasis web dengan integrasi notifikasi melalui Bot Telegram dan pencatatan log historis. Menggunakan metode Prototype yang mencakup analisis kebutuhan, perancangan, implementasi prototype, pengujian, dan penyempurnaan sistem.	Sistem mampu memantau status perangkat secara real-time, mencatat insiden jaringan, serta mengirimkan notifikasi otomatis ke Telegram saat terjadi gangguan seperti perangkat down, insiden aktif, dan perubahan log. Sistem juga menyediakan validasi manual dan log historis untuk analisis dan evaluasi kinerja jaringan. Sehingga respons admin dan evaluasi jaringan menjadi lebih efektif (Baydowi & Al Islami, 2026).

Tabel 2.6 State of The Art (Lanjutan)

No.	Penelitian	Metode	Hasil Penelitian
9.	Judul: Design and Implementation of Network Monitoring System Using Zabbix and Telegram Penulis: Prasetyo Fajar Malik, Bony Parulian Josapha Tahun: 2024	Mendesain dan mengimplementasikan sistem monitoring jaringan menggunakan Zabbix dengan integrasi Telegram Bot serta protokol SNMP dan webhook untuk pengiriman notifikasi. Menggunakan metode NDLC (Analysis, Design, Simulation, Implementation, Monitoring, Management).	Sistem mampu memonitor sumber daya data center secara real-time (CPU, memori, traffic) dan mengirim notifikasi otomatis ke Telegram saat terjadi gangguan seperti interface down, latensi tinggi, dan perubahan status perangkat. Notifikasi juga dikirim saat kondisi kembali normal untuk mendukung respons cepat administrator (Malik & Josaphat, 2024).
10.	Judul: Implementasi Monitoring Manajemen Jaringan Dengan Software The Dude Berbasis Telegram Messenger Penulis: Sintiya Puji Rahayu, I Gusti Lanang Putra Eka Prisma Tahun: 2022 (Rahayu & Prisma, 2022)	Mengimplementasikan sistem monitoring jaringan menggunakan The Dude pada Mikrotik dengan integrasi Telegram Bot API. Metode meliputi analisis kebutuhan, perancangan sistem monitoring, konfigurasi SNMP, serta pengujian notifikasi melalui Telegram.	Sistem mampu memonitor perangkat jaringan secara real-time dan mengirim notifikasi otomatis ke Telegram saat terjadi perubahan status perangkat (down/up). Notifikasi memuat waktu, nama perangkat, alamat IP, dan status layanan untuk mempercepat troubleshooting.

Berdasarkan Tabel 2.2, sebagian besar penelitian terdahulu dalam implementasi sistem monitoring jaringan berfokus pada kemampuan menampilkan parameter jaringan dan memberikan notifikasi otomatis saat terjadi perubahan kondisi. Namun, pendekatan yang digunakan masih terbatas pada mekanisme *threshold-based alert* atau notifikasi berbasis perubahan status (misalnya *up/down*), yang sering kali mengandalkan konfigurasi *default* atau *template* dari tools monitoring seperti Zabbix atau fitur bawaan MikroTik. Selain itu, parameter yang dianalisis cenderung diperlakukan secara terpisah, seperti *CPU utilization*, *traffic*, atau status *interface*, tanpa mempertimbangkan keterkaitan antar parameter dalam satu konteks gangguan.

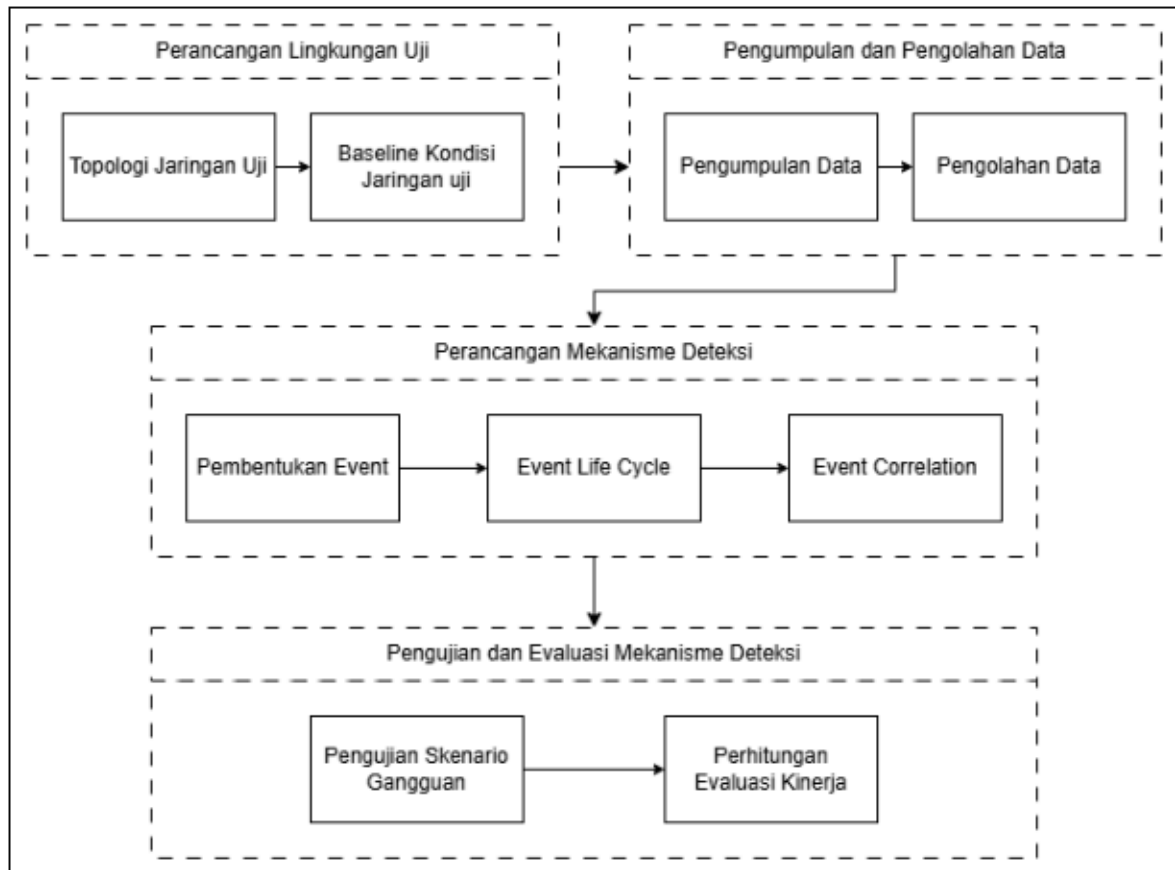
Keterbatasan ini menyebabkan informasi yang dihasilkan bersifat parsial karena setiap notifikasi berdiri sendiri. Akibatnya, administrator perlu melakukan analisis manual untuk memahami penyebab gangguan, sehingga proses identifikasi menjadi kurang efisien.

Untuk mengatasi hal tersebut, penelitian ini mengusulkan pendekatan dengan menggabungkan *rule-based* dan *event correlation*. *Rule-based* digunakan untuk membentuk *event* berdasarkan kondisi tertentu, sedangkan *event correlation* menghubungkan beberapa *event* yang terjadi dalam rentang waktu berdekatan. Pendekatan ini memungkinkan sistem tidak hanya menampilkan kondisi jaringan, tetapi juga memberikan indikasi awal yang lebih terarah terhadap kemungkinan gangguan.

BAB III METODOLOGI PENELITIAN

3.1 Metodologi Penelitian

Tahapan penelitian ini berfokus pada implementasi sistem monitoring berbasis MikroTik dengan pendekatan *rule-based* dan *event correlation* untuk menganalisis indikasi gangguan jaringan. Tahapan penelitian ditunjukkan pada Gambar 3.1.



Gambar 3.1 Tahapan Penelitian

3.1.1 Perancangan Lingkungan Uji

Tahap pertama dalam penelitian ini adalah perancangan lingkungan uji untuk menyediakan kondisi eksperimen yang terkontrol dan dapat direplikasi. Lingkungan uji dirancang dalam bentuk simulasi jaringan yang mencakup dua aspek utama, yaitu perancangan topologi jaringan dan penetapan baseline kondisi jaringan sebagai acuan dalam pelaksanaan pengujian. Secara keseluruhan, tahap perancangan lingkungan uji dijabarkan sebagai berikut.

1. Perancangan Topologi Jaringan Uji

Topologi jaringan uji pada penelitian ini disusun dalam bentuk simulasi jaringan menggunakan GNS3 dengan MikroTik Cloud Hosted Router (CHR) sebagai perangkat utama yang dimonitor. Selain CHR, digunakan beberapa Ubuntu Server yang berfungsi sebagai *upstream emulator*, generator gangguan jaringan, serta

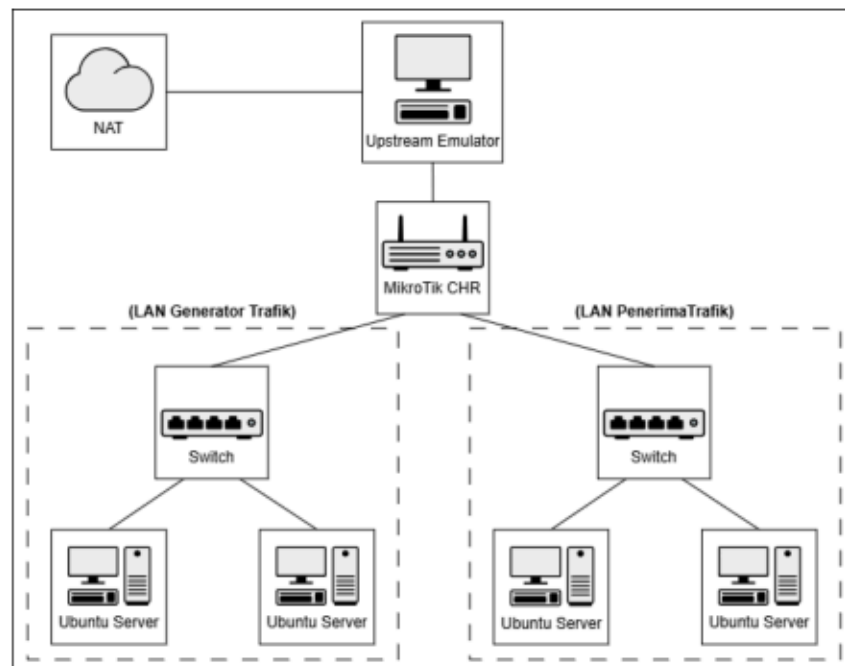
penerima trafik. Versi, spesifikasi, dan fungsi masing-masing komponen disajikan pada Tabel 3.1.

Tabel 3.1 Spesifikasi Setiap Node Pada Topologi

Komponen	Versi	Spesifikasi	Fungsi
GNS3	2.2.58.1	-	Menjalankan topologi serta <i>deployment</i> MikroTik CHR dan VM
MikroTik CHR	7.20.8	1 vCPU, 1 GB RAM	Router utama dan objek monitoring
Generator gangguan jaringan (Ubuntu Server)	22.04 LTS (Jammy)	1 vCPU, 1GB RAM	Menghasilkan trafik/gangguan menuju jaringan penerima
Penerima trafik (Ubuntu Server)	22.04 LTS (Jammy)	1 vCPU, 1GB RAM	Menerima trafik dari generator
Upstream emulator (Ubuntu Server)	22.04 LTS (Jammy)	1 vCPU, 1GB RAM	Memmanipulasi kualitas jaringan (<i>delay</i> dan <i>packet loss</i>)

Setiap komponen pada Tabel 3.1 dirancang agar seluruh trafik melewati jalur yang sama sehingga kondisi jaringan dapat dikontrol. GNS3 sebagai host terhubung ke jaringan NAT, kemudian trafik diteruskan ke *upstream emulator* sebelum mencapai MikroTik CHR. Penempatan *upstream emulator* ini bertujuan untuk memanipulasi kualitas jaringan, seperti *delay* dan *packet loss*, sesuai kebutuhan pengujian.

Pada sisi *downstream*, CHR terhubung ke *switch* yang membagi jaringan menjadi dua segmen, yaitu jaringan generator gangguan dan jaringan penerima trafik. Dengan desain ini, trafik yang dihasilkan oleh node generator akan melewati CHR sebelum mencapai node penerima. Secara keseluruhan, desain topologi jaringan uji ditunjukkan pada Gambar 3.2.



Gambar 3.2 Desain Topologi Lingkungan Uji

2. Perancangan Baseline Kondisi Jaringan Uji

Setelah topologi jaringan uji ditetapkan, langkah selanjutnya adalah menentukan baseline kondisi jaringan sebagai acuan awal sebelum skenario gangguan diterapkan. Baseline ini digunakan untuk mendefinisikan kondisi jaringan secara konsisten, sehingga kondisi normal dan perubahan performa akibat degradasi layanan dapat dibedakan.

Penetapan baseline pada penelitian ini mengacu pada standar ITU-T Y.1541 dengan menggunakan Class 2 sebagai acuan kondisi layanan. Pemilihan kelas ini didasarkan pada trafik pengujian berupa ICMP (*ping*) antara MikroTik CHR dan *upstream emulator* yang termasuk trafik kontrol jaringan (*signaling* sederhana) untuk evaluasi *delay* dan *packet loss*, sehingga sesuai dengan kategori penggunaan Class 2 pada Tabel 2.1.

Nilai pada Class 2 kemudian diklasifikasikan menjadi *good*, *medium*, dan *poor*. Kategori *good* merepresentasikan kondisi normal yang masih memenuhi standar, *medium* menunjukkan penurunan performa yang masih dapat ditoleransi, sedangkan *poor* mengindikasikan kondisi yang berpotensi mengganggu jaringan. Klasifikasi ini digunakan sebagai referensi dalam menginterpretasikan kondisi jaringan selama pengujian. Secara keseluruhan, baseline kondisi jaringan uji pada penelitian ini disajikan pada Tabel 3.2.

Tabel 3.2 Baseline Kondisi Jaringan Uji

Kondisi	Packet Loss	Delay (RTT)
Good (Ideal)	$\leq 0.1\%$	$\leq 200\text{ ms}$
Medium (Sedang)	$0.1\% \text{ s/d } 1\%$	$200\text{ ms s/d } 400\text{ ms}$
Poor (Buruk)	$> 1\%$	$> 400\text{ ms}$

3.1.2 Pengumpulan dan Pengolahan Data

Tahap pengumpulan dan pengolahan data dilakukan dengan mengumpulkan data parameter kesehatan jaringan dari perangkat MikroTik berdasarkan topologi dan *baseline* kondisi jaringan yang telah dirancang. Data yang diperoleh kemudian diolah agar siap untuk dianalisis lebih lanjut. Secara keseluruhan, tahap pengumpulan dan pengolahan data dijabarkan sebagai berikut.

1. Pengumpulan Data

Tahap pengumpulan data dilakukan menggunakan RouterOS API untuk memperoleh parameter kesehatan jaringan dari perangkat MikroTik. Parameter kesehatan yang diperoleh dibedakan menjadi dua jenis, yaitu data statis dan data dinamis.

Data statis merupakan identitas perangkat yang sifatnya tidak mengalami perubahan selama sistem berjalan sehingga hanya diambil saat sistem pertama kali terhubung dengan perangkat. Parameter yang termasuk dalam kategori ini meliputi nama perangkat, alamat IP, versi RouterOS, arsitektur sistem, serta informasi model perangkat.

Data dinamis merupakan parameter kesehatan jaringan yang nilainya dapat berubah sesuai dengan kondisi jaringan dan perangkat. Oleh karena itu, parameter

ini diambil secara periodik menggunakan mekanisme polling agar perubahan kondisi jaringan dapat terpantau secara berkelanjutan. Parameter yang termasuk dalam kategori ini meliputi performa perangkat, konektivitas interface, serta parameter kualitas layanan jaringan.

Pengambilan data dinamis dilakukan secara periodik dengan interval minimal 5 detik untuk menjaga keseimbangan antara respons sistem dan beban pemrosesan. Setiap siklus dijalankan setelah proses sebelumnya selesai guna menghindari *overlap*. Berikut merupakan parameter data dinamis yang diperoleh dari perangkat MikroTik beserta *endpoint*-nya.

a. Parameter Performa Perangkat

Parameter performa perangkat meliputi penggunaan CPU, penggunaan memori, dan uptime. Data diperoleh melalui endpoint *GET /system/resource/*, dengan field yang diambil berupa *cpu-load*, *total-memory*, *free-memory*, dan *uptime*. Contoh respons ditunjukkan pada Contoh Data 1.

Contoh Data 1. Respons endpoint *GET /system/resource/*

```
{
  "cpu-load": "37 %",
  "total-memory": "1024.0 MiB",
  "free-memory": "855.5 MiB",
  "uptime": "2h14m51s"
}
```

b. Parameter Konektivitas Interface

Parameter konektivitas interface meliputi status dan identitas setiap interface. Data diperoleh melalui endpoint *GET /interface/*, dengan field yang diambil berupa *.id*, *name*, *type*, *running*, dan *disabled*. Respons dari endpoint tersebut ditunjukkan pada Contoh Data 2.

Contoh Data 2. Respons endpoint *GET /interface/*

```
[
  {
    ".id": "*1"
    name: "ether1",
    type: "ether",
    running: "true",
    disabled: "false"
  },
  {
    ".id": "*2"
    name: "ether2",
    type: "ether",
    running: "true",
    disabled: "false"
  },
]
```

c. Parameter Kualitas Layanan Jaringan

Parameter kualitas layanan jaringan meliputi *delay* (RTT) dan *packet loss* yang diperoleh melalui pengujian *ping* menggunakan endpoint *POST /tool/ping* dengan *request body count = 10* dan *interval = "450ms"*. Penambahan *interval* 450 ms bertujuan mengurangi kemungkinan *timeout* pada kondisi jaringan dengan *delay* tinggi. Data yang diambil meliputi *sent*, *received*, dan *time*. Respons endpoint ditunjukkan pada Kode 3.

Contoh Data 3. Respons endpoint *POST /tool/ping*

```
[
  {
    "sent": "9",
    "received": "8",
    "time": "29ms81us"
  },
  {
    "sent": "10",
    "received": "9",
    "time": "28ms76us"
  },
  ...
]
```

2. Pengolahan Data

Setelah data parameter kesehatan jaringan diperoleh dari perangkat MikroTik, dilakukan tahap pengolahan untuk menyesuaikan format dan struktur data agar siap digunakan dalam analisis gangguan jaringan yang ada melalui mekanisme *rule-based detection* dan *event correlation*. Pengolahan data terdiri dari dua tahap utama, yaitu penyesuaian struktur dan format data serta pembentukan dan perhitungan statistik *sliding window*.

a. Restrukturisasi dan Normalisasi Data

Tahap ini bertujuan menyeragamkan data agar dapat diproses secara konsisten. Proses yang dilakukan meliputi penyesuaian nama atribut, konversi nilai numerik dari *string* ke tipe numerik, serta konversi nilai *Boolean* dalam bentuk *string* menjadi tipe *boolean*.

Selain itu, pada tahap ini juga dilakukan penambahan *timestamp* dalam satuan milidetik untuk setiap data, serta perhitungan nilai turunan guna mempermudah interpretasi data, seperti persentase penggunaan memori, jumlah memori tersedia, dan memori terpakai. Contoh hasil pengolahan data ditunjukkan pada Contoh Data 4 hingga Contoh Data 8.

Contoh Data 4. Penyesuaian struktur data penggunaan CPU

```
{  
  ts: 1718024990000,  
  cpuLoad: 34.5  
}
```

Contoh Data 5. Penyesuaian struktur data penggunaan memori

```
{  
  ts: 1718024990000,  
  totalMem: 1024  
  freePct: 79,  
  freeMem: 803.8,  
  usedPct: 21,  
  usedMem: 220.2  
}
```

Contoh Data 6. Penyesuaian struktur data delay jaringan

```
{  
  ts: 1718024995000,  
  avgDelayMs: 27.7  
}
```

Contoh Data 7. Penyesuaian struktur data packet loss

```
{  
  ts: 1718024995000,  
  packetSent: 10,  
  packetReceived: 9  
}
```

Contoh Data 8. Penyesuaian struktur data status dan throughput interface

```
[  
  {  
    id: "*1"  
    name: "ether1",  
    type: "ether",  
    running: true,  
    disabled: false  
  },  
  {  
    id: "*2"  
    name: "ether2",  
    type: "ether",  
    running: true,  
    disabled: false  
  },  
  ...  
]
```

b. Pembentukan sliding window

Data yang telah diproses kemudian dimasukkan ke dalam struktur sliding window, yaitu kumpulan data historis dalam rentang waktu tertentu. Setiap data

baru akan ditambahkan, sementara data lama akan dihapus ketika kapasitas maksimum tercapai.

Pendekatan ini memungkinkan analisis dilakukan berdasarkan sekumpulan data terbaru dalam interval waktu tertentu, sehingga hasilnya lebih stabil dan tidak terpengaruh fluktuasi sesaat.

Parameter yang disimpan dalam window meliputi CPU, memori, delay, dan packet loss. Struktur sliding window ditunjukkan pada Contoh Data 9 hingga Contoh Data 12.

Contoh Data 9. Struktur sliding window penggunaan CPU

```
cpuWindow = [
  { ts:_, loadPct:_ },
  { ts:_, loadPct:_ },
  ...
]
```

Contoh Data 10. Struktur sliding window penggunaan memori

```
memoryWindow = [
  { ts:_, usedPct:_, usedMb:_, freePct:_,freeMb:_ },
  { ts:_, usedPct:_, usedMb:_, freePct:_,freeMb:_ },
  ...
]
```

Contoh Data 11. Struktur sliding window delay jaringan

```
latencyWindow = [
  { ts:_, avgMs:_ },
  { ts:_, avgMs:_ },
  ...
]
```

Contoh Data 12. Struktur sliding window packetloss

```
packetLossWindow = [
  { ts:_, sent:_, received:_ },
  { ts:_, sent:_, received:_ },
  ...
]
```

c. Perhitungan Statistik Window

Data yang telah tersimpan dalam *sliding window* kemudian diolah untuk menghasilkan nilai statistik yang mewakili kondisi jaringan dalam periode tertentu. Pada parameter CPU, memori, dan delay, nilai yang dihitung meliputi minimum, maksimum, dan rata-rata dari seluruh data dalam *window*. Sementara itu, pada parameter *packet loss* dilakukan agregasi dengan menjumlahkan nilai *sent* dan *received*, kemudian digunakan untuk menghitung total kehilangan paket serta persentase *packet loss*. Pendekatan ini menghasilkan representasi kondisi jaringan yang lebih stabil dan tidak bergantung pada satu kali pengukuran, sehingga lebih akurat untuk digunakan dalam proses analisis selanjutnya. Data

yang dihasilkan tersebut selanjutnya dapat digunakan sebagai dasar analisis *rule-based* dan *event correlation* serta visualisasi data.

3.1.3 Perancangan Mekanisme Deteksi

Mekanisme deteksi gangguan jaringan pada penelitian ini terdiri dari tiga tahapan utama. Proses diawali dengan pembentukan event menggunakan pendekatan *rule-based detection*. Selanjutnya, event yang terbentuk dikelola melalui event lifecycle untuk menggambarkan perubahan kondisi. Tahap berikutnya adalah event correlation, yaitu menghubungkan beberapa event dalam rentang waktu tertentu untuk membentuk indikasi awal gangguan jaringan.

Ketiga tahapan tersebut disusun secara berurutan agar proses deteksi mempertimbangkan keterkaitan antar event, tidak hanya berdasarkan satu kondisi tunggal. Penjelasan lebih lanjut mengenai masing-masing tahapan disajikan pada subbagian berikut.

1. Mekanisme Pembentukan Event

Tahapan pembentukan *event* dilakukan menggunakan pendekatan *rule-based detection* yang mempertimbangkan dua komponen utama, yaitu *threshold* dan rentang waktu pengamatan. Suatu *event* terbentuk apabila nilai rata-rata parameter dalam suatu rentang waktu (*sliding window*) memenuhi atau melampaui *threshold* yang ditetapkan. Dengan demikian, nilai parameter tidak harus selalu berada di atas *threshold* pada setiap pengukuran, selama rata-ratanya dalam periode tersebut masih berada pada atau di atas batas yang ditentukan.

Pendekatan ini digunakan untuk mengurangi pengaruh fluktuasi sesaat serta menghasilkan deteksi yang lebih merepresentasikan kondisi jaringan terkini. Daftar rule yang digunakan dalam penelitian ini disajikan pada Tabel 3.3.

Tabel 3.3 Daftar Rule Pembentukan Event

ID	Parameter	Threshold dan Durasi	Event
R1	Status Interface	Perubahan status interface dari UP menjadi DOWN	Interface Down
R2	Delay (RTT)	Rata-rata RTT selama 60 $\geq$ 250 ms	Delay Tinggi
R3	Packet Loss	Rata-rata packet loss selama 60 detik $\geq$ 3%	Packet Loss Tinggi
R4	Penggunaan CPU	Rata-rata penggunaan CPU selama 60 detik $\geq$ 80%	Beban CPU Berlebih

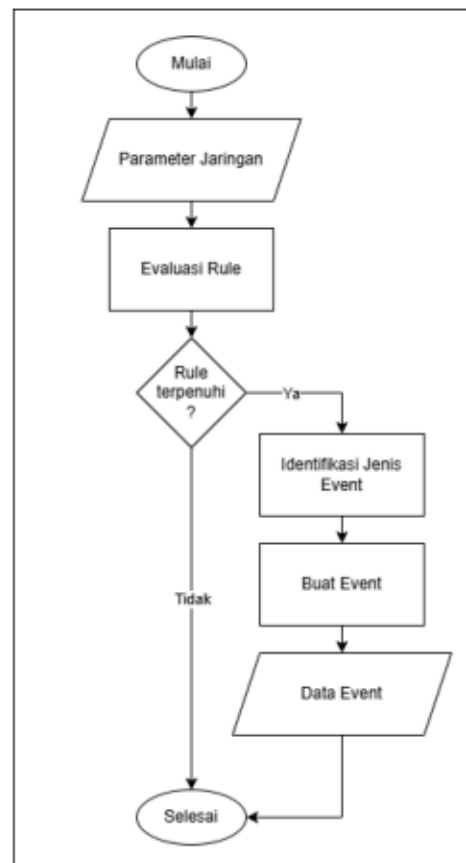
Pemilihan nilai *threshold* pada setiap parameter pada Tabel 3.3 disesuaikan dengan karakteristik metrik serta kondisi baseline jaringan pada lingkungan pengujian, sebagaimana dijelaskan pada Subbab 2.2.1. Parameter status interface digunakan sebagai indikator langsung karena perubahan ke kondisi DOWN secara jelas menunjukkan terputusnya konektivitas.

Parameter QoS seperti delay dan packet loss tidak memiliki standar baku nilai *threshold* yang bersifat universal, melainkan bergantung pada jenis layanan dan kondisi jaringan yang digunakan. Oleh karena itu, pada penelitian ini *threshold* delay

dan packet loss ditentukan berdasarkan baseline yang telah dimodelkan sebelumnya, sehingga lebih sesuai dalam membedakan antara kondisi normal dan degradasi layanan pada skenario pengujian. Sementara itu, penggunaan CPU $\geq 80\%$ digunakan sebagai batas umum yang banyak diterapkan pada sistem monitoring.

Pemilihan durasi minimal pelanggaran selama 60 detik pada parameter berbasis *sliding window* dilakukan untuk mengurangi sensitivitas deteksi terhadap fluktuasi sesaat (*transient spike*) yang belum tentu merepresentasikan gangguan aktual. Nilai ini mengadaptasi praktik monitoring jaringan pada penelitian terdahulu yang menggunakan interval monitoring 60 detik untuk memperoleh kondisi jaringan yang lebih konsisten (Caicedo-Altamirano, Villacrés-Cevallos, et al., 2025). Pada penelitian ini, durasi tersebut digunakan sebagai periode evaluasi pelanggaran *threshold*, sehingga dengan interval *polling* 5 detik diperoleh sekitar 12 sampel pengamatan yang dinilai lebih representatif terhadap kondisi jaringan.

Ketika suatu rule terpenuhi, sistem akan menghasilkan event yang merepresentasikan kondisi gangguan sesuai parameter yang terdeteksi. Pada saat yang sama, evidence berupa nilai parameter yang memicu event tersebut dicatat sebagai konteks pendukung. Seluruh informasi ini kemudian disimpan untuk digunakan pada proses analisis selanjutnya. Secara keseluruhan, alur pembentukan event dengan pendekatan rule-based detection ditunjukkan pada Gambar 3.3.



Gambar 3.3 Flowchart Mekanisme Pembentukan Event

2. Event Life Cycle

Mekanisme event life cycle bertujuan menjaga konsistensi deteksi dengan mencegah event berakhir akibat fluktuasi sesaat serta menghindari terbentuknya beberapa event untuk gangguan yang sama. Dengan demikian, satu gangguan dapat direpresentasikan sebagai satu event yang utuh.

Dalam mekanisme ini, setiap event merepresentasikan kondisi satu parameter kesehatan jaringan pada suatu perangkat dalam satu waktu. Oleh karena itu, pembentukan event baru dengan karakteristik yang sama hanya dapat dilakukan apabila event sebelumnya telah berstatus *ended*. Sebagai contoh, event *CPU overload* merepresentasikan kondisi penggunaan CPU yang sedang tidak normal, sehingga tidak dimungkinkan terdapat lebih dari satu event aktif dengan parameter pembentuk yang sama pada perangkat yang sama secara bersamaan. Khusus pada parameter berbasis interface, pembatasan ini diterapkan pada masing-masing interface, sehingga satu interface hanya dapat memiliki satu event aktif dengan karakteristik yang sama pada satu waktu.

Setelah suatu event terbentuk, status awalnya adalah *active*. Pada tahap ini, mekanisme *event life cycle* melakukan pengecekan secara periodik terhadap event yang berstatus *active* maupun *cooldown*. Event dengan status *active* akan dilakukan pengecekan untuk memastikan apakah kondisi masih memenuhi *threshold*. Jika masih terpenuhi, status tetap *active* dan nilai *last seen* diperbarui sebagai penanda waktu terakhir gangguan terdeteksi. Sebaliknya, jika tidak lagi memenuhi *threshold*, event beralih ke status *cooldown*. Status *cooldown* bertujuan untuk memastikan bahwa kondisi jaringan telah kembali normal secara stabil, bukan akibat fluktuasi sesaat. Pada tahap ini ditetapkan nilai *cooldown until*, yaitu waktu saat perubahan status ke *cooldown* ditambah 90 detik, yang berasal dari durasi *time window* selama 60 detik dengan tambahan toleransi 30 detik.

Event dengan status *cooldown* akan dilakukan pengecekan untuk memastikan apakah kondisi kembali memenuhi *threshold*. Jika sebelum waktu *cooldown* berakhir kondisi kembali memenuhi *threshold*, status dikembalikan menjadi *active* dan nilai *last seen* diperbarui. Sebaliknya, jika tidak terjadi pelanggaran hingga batas waktu tersebut, event diakhiri dengan status *ended*.

3. Event Correlation

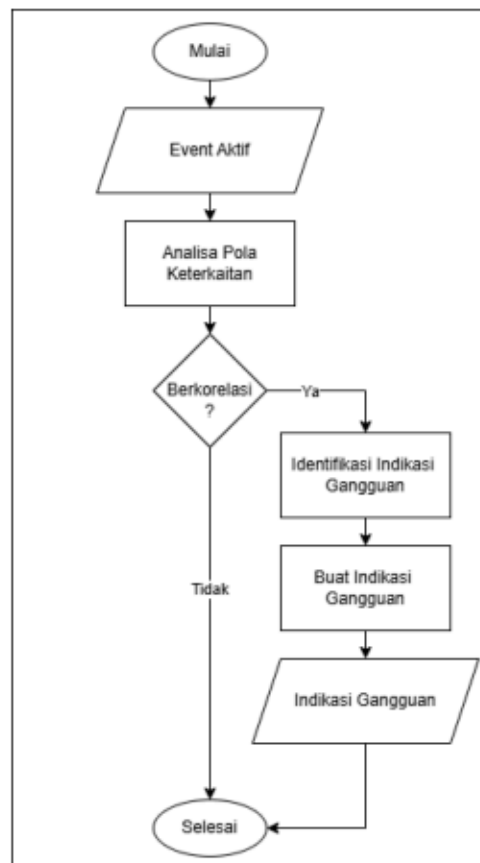
Tahap *event correlation* merupakan proses lanjutan setelah pembentukan *event* melalui pendekatan *rule-based detection*. Pada tahap ini, beberapa *event* yang memiliki keterkaitan dikaitkan berdasarkan keterkaitan antar parameternya maupun kedekatan waktu kemunculannya untuk membentuk indikasi awal gangguan jaringan. Daftar pola korelasi yang digunakan dalam penelitian ini disajikan pada Tabel 3.4.

Tabel 3.4 Daftar Pola Indikasi Jaringan

ID	Pola Event yang Dikorelasikan	Indikasi Awal Gangguan
C1	Delay dan packet loss tinggi dalam rentan waktu 90 detik tanpa beban CPU berlebih	Kemungkinan degradasi kualitas jaringan pada sisi upstream atau ISP
C2	Delay dan packet loss tinggi dengan beban CPU berlebih dalam rentan waktu 90 detik	Kemungkinan degradasi kualitas jaringan yang akibat keterbatasan resource perangkat

Pola korelasi pada Tabel 3.4 disusun berdasarkan keterkaitan antar parameter pembentuk event, sebagaimana dijelaskan pada Subbab 2.2.2. Rentang waktu 90 detik digunakan untuk memastikan kedekatan waktu kemunculan event, sehingga event yang tidak berdekatan tidak dikorelasikan. Nilai ini ditentukan berdasarkan durasi time window 60 detik dengan tambahan toleransi 30 detik.

Ketika pola korelasi terpenuhi, maka dihasilkan indikasi gangguan jaringan beserta event pembentuknya. Secara keseluruhan, alur event correlation ditunjukkan pada Gambar 3.4.



Gambar 3.4 Flowchart Proses Event Correlation

3.1.4 Pengujian dan Evaluasi Kinerja Mekanisme Deteksi

Tahap pengujian dan evaluasi dilakukan untuk menilai kinerja mekanisme deteksi gangguan jaringan yang telah dirancang. Pengujian dilakukan untuk mengamati respons sistem terhadap skenario gangguan yang diberikan, sedangkan evaluasi dilakukan untuk menilai kinerja sistem berdasarkan hasil pengujian. Secara keseluruhan, tahapan pengujian dan evaluasi dijabarkan sebagai berikut.

1. Pengujian Skenario Gangguan

Tahap pengujian pada penelitian ini mencakup pembentukan event oleh *rule-based detection*, perubahan status melalui *event lifecycle (active-cooldown-ended)*, validasi bahwa sistem tidak menghasilkan event pada kondisi normal, serta pembentukan indikasi gangguan melalui *event correlation*.

Pengujian dilakukan pada satu perangkat MikroTik CHR dalam satu sesi monitoring dengan menggunakan beberapa skenario gangguan yang dirancang untuk merepresentasikan kondisi jaringan tertentu. Pemilihan skenario tersebut bertujuan untuk memfokuskan evaluasi pada kemampuan mekanisme *rule-based detection* dan *event correlation* dalam membentuk event serta menghasilkan indikasi awal gangguan jaringan secara terkontrol.

Sebelum pengujian dilakukan, topologi jaringan dipastikan dalam keadaan aktif dan seluruh perangkat dapat saling berkomunikasi. Secara keseluruhan, skenario pengujian yang digunakan dalam penelitian ini disajikan pada Tabel 3.5.

Tabel 3.5 Skenario Pengujian

Unit Uji	ID	Skenario Pengujian	Hasil yang Diharapkan
R1	S1	Menonaktifkan interface secara manual pada CHR	Event Interface Down terbentuk beserta evidence kondisi saat pembentukan
	S2	Mengaktifkan kembali interface	Status event berubah menjadi ended
R2	S3	Meningkatkan delay menggunakan tc netem hingga melebihi threshold dan mempertahankannya lebih lama dari durasi pelanggaran	Event Delay Tinggi terbentuk beserta evidence kondisi saat pembentukan
	S4	Mengembalikan delay ke kondisi normal hingga melewati durasi cooldown	Status event berubah menjadi cooldown, kemudian menjadi ended setelah durasi cooldown terpenuhi
	S5	Meningkatkan kembali delay hingga melewati threshold sebelum durasi cooldown berakhir	Status event kembali menjadi active dan kejadian tetap dianggap satu event
	S6	Menjaga kondisi delay dalam batas normal (tidak melebihi threshold)	Event tidak terbentuk
R3	S7	Menginjeksi packet loss menggunakan tc netem hingga melebihi threshold dan mempertahankannya lebih lama dari durasi pengamatan lebih lama dari durasi pelanggaran	Event Packet Loss Tinggi terbentuk beserta evidence kondisi saat pembentukan
	S8	Menghentikan packet loss hingga melewati durasi cooldown	Status event berubah menjadi cooldown, kemudian menjadi ended setelah durasi cooldown terpenuhi
	S9	Memunculkan kembali packet loss hingga melewati threshold sebelum durasi cooldown berakhir	Status event kembali menjadi active dan kejadian tetap dianggap satu event
	S10	Menjaga packet loss dalam kondisi normal (tidak melebihi threshold)	Event tidak terbentuk

Tabel 3.6 Skenario Pengujian (Lanjutan)

Unit Uji	ID	Skenario Pengujian	Hasil yang Diharapkan
R4	S11	Meningkatkan beban CPU menggunakan hping (flood traffic) hingga melebihi threshold dan mempertahankannya lebih lama dari durasi pelanggaran	Event Beban CPU Berlebih terbentuk beserta evidence kondisi saat pembentukan
	S12	Menghentikan beban trafik hingga CPU stabil melewati durasi cooldown	Status event berubah menjadi cooldown, kemudian menjadi ended setelah durasi cooldown terpenuhi
	S13	Meningkatkan kembali beban CPU hingga melewati threshold sebelum durasi cooldown berakhir	Status event kembali menjadi active dan kejadian tetap dianggap satu event
	S14	Menjaga beban CPU dalam kondisi normal (tidak melebihi threshold)	Event tidak terbentuk
C1	S15	Membentuk event QoS tanpa event CPU dalam rentang waktu korelasi	Indikasi degradasi kualitas jaringan tanpa keterlibatan resource perangkat (C1) terbentuk dan relasi antar event tersimpan
	S16	Mengakhiri kondisi QoS (delay dan packet loss)	Status indikasi berubah menjadi ended dan waktu selesai indikasi tercatat
	S17	Meningkatkan hanya salah satu parameter QoS (delay atau packet loss) hingga membentuk event	Indikasi C1 tidak terbentuk
C2	S18	Membentuk event QoS dan event CPU dalam rentang waktu korelasi	Indikasi degradasi kualitas jaringan akibat keterbatasan resource perangkat (C2) terbentuk dan relasi antar event tersimpan
	S19	Mengakhiri event QoS dan CPU	Status indikasi berubah menjadi ended dan waktu selesai indikasi tercatat
	S20	Membentuk event CPU di luar rentang waktu korelasi dari event QoS	Indikasi C2 tidak terbentuk, sistem hanya mempertahankan indikasi C1

2. Evaluasi

Tahap evaluasi dilakukan dengan menghitung parameter *Mean Time To Detect* (MTTD) dan *Detection Rate* (DR) dari mekanisme deteksi pada sistem monitoring yang dikembangkan berdasarkan hasil pengujian sebelumnya. Pengukuran kedua parameter tersebut disimulasikan sebagai berikut.

a. Simulasi Perhitungan MTTD

Perhitungan MTTD dilakukan dengan menghitung selisih antara waktu dimulainya gangguan dan waktu gangguan terdeteksi oleh sistem pada setiap pengujian yang menghasilkan event atau indikasi gangguan.

Sebagai ilustrasi, dilakukan 12 pengujian gangguan yang menghasilkan event atau indikasi. Waktu dimulainya gangguan dan waktu terdeteksinya

dicatat untuk dihitung selisihnya. Pencatatan data MTTD ditampilkan pada Tabel 3.7.

Tabel 3.7 Simulasi Perhitungan MTTD

ID Pengujian	Waktu Gangguan Dimulai	Waktu Gangguan Terdeteksi	Selisih Waktu (detik)
S1	15.00.00	15.00.10	10
	15.00.20	15.00.33	13
S3	15.01.00	15.01.07	7
	15.01.15	15.01.21	6
S7	15.02.00	15.02.09	9
	15.02.20	15.02.30	10
S11	15.03.00	15.03.12	12
	15.03.20	15.03.31	11
S14	15.04.00	15.04.08	8
	15.04.15	15.04.24	9
S17	15.05.00	15.05.14	14
	15.05.20	15.05.33	13

Data dari Tabel 3.7 kemudian digunakan untuk menghitung nilai MTTD sesuai dengan Persamaan (2.1), sehingga diperoleh nilai MTTD sebagai berikut :

$$MTTD = \frac{10 + 13 + 7 + 6 + 9 + 10 + 12 + 11 + 8 + 9 + 14 + 13}{12} = 10,17 \text{ detik}$$

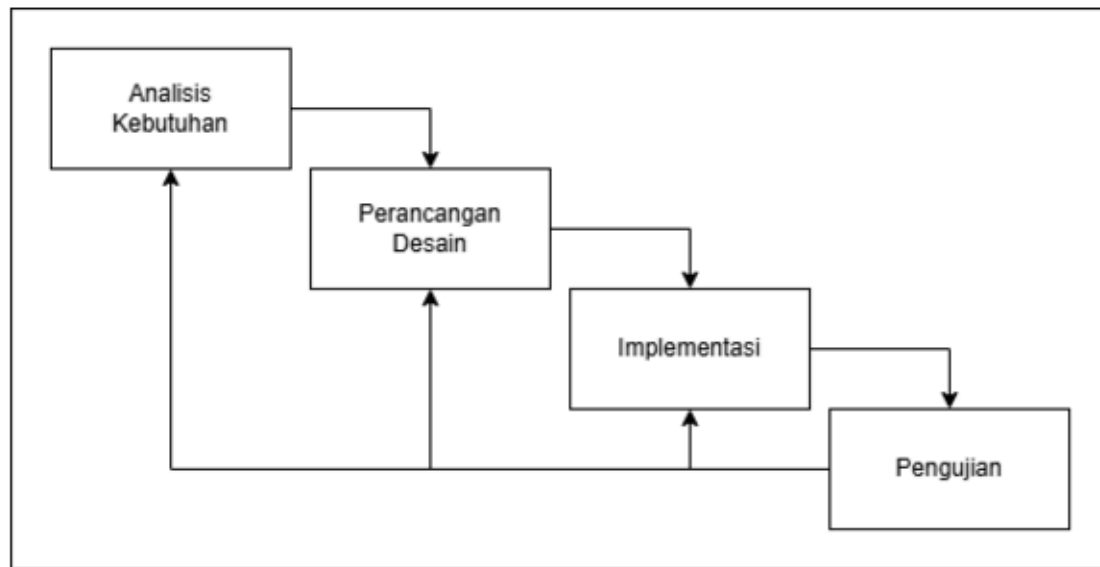
b. Simulasi Perhitungan DR

Perhitungan DR dilakukan dengan mencatat jumlah gangguan yang berhasil terdeteksi dengan benar pada setiap pengujian, kemudian dibandingkan dengan total pengujian. Misalnya, dari 12 pengujian gangguan terdapat 11 yang terdeteksi dengan benar, data ini kemudian digunakan untuk menghitung DR sesuai Persamaan (2.2), sehingga diperoleh nilai DR sebagai berikut:

$$DR = \frac{11}{12} \times 100\% = 91\%$$

3.2 Metodologi Pengembangan Sistem

Penelitian ini menggunakan metode pengembangan sistem *waterfall* sebagai model *System Development Life Cycle* (SDLC) dalam mengembangkan sistem monitoring. Pendekatan *waterfall* pada penelitian ini dibatasi hingga tahap pengujian dan tidak mencakup tahap pemeliharaan untuk menjaga fokus dan ruang lingkup penelitian. Tahapan pengembangan sistem yang dilakukan ditunjukkan pada gambar 3.5



Gambar 3.5 Metode Waterfall

3.2.1 Analisis Kebutuhan

Tahapan pertama pada pengembangan sistem monitoring ini yaitu analisis kebutuhan. Tahapan ini bertujuan untuk mendefinisikan kebutuhan fungsional dan non fungsional sistem monitoring yang akan dikembangkan.

1. Kebutuhan Fungsional

Kebutuhan fungsional mengacu pada kemampuan atau fungsi yang harus dimiliki oleh sistem dalam menjalankan proses tertentu. Dalam penelitian ini, kebutuhan fungsional sistem monitoring yang akan dikembangkan meliputi hal-hal berikut:

- Sistem mampu menerima data autentikasi dan terhubung dengan perangkat MikroTik.
- Sistem mampu mengambil parameter jaringan dari perangkat MikroTik secara periodik dan menampilkannya pada dashboard monitoring.
- Sistem mampu mendeteksi dan membentuk event beserta siklus hidupnya berdasarkan threshold dan durasi yang ditetapkan.
- Sistem mampu menganalisis indikasi gangguan jaringan berdasarkan korelasi antar event sesuai waktu kejadian dan keterkaitan parameternya.

2. Kebutuhan Non Fungsional

Kebutuhan non fungsional mencakup spesifikasi minimum yang diperlukan untuk mengimplementasikan sistem yang dikembangkan. Kebutuhan tersebut meliputi aspek perangkat keras dan perangkat lunak. Berikut ini adalah analisis kebutuhan non fungsional pada sistem monitoring yang dikembangkan dalam penelitian ini:

a. Analisis Perangkat Keras

Spesifikasi perangkat keras yang digunakan dalam penelitian ini disajikan pada Tabel 3.8

Tabel 3.8 Spesifikasi Perangkat Keras

Komponen	Spesifikasi
Processor(CPU)	Intel Core i5-11300H (4 Cores, 8 Threads, 4.4 GHz)
Graphics Card (GPU)	NVIDIA GeForce GTX 1650 4GB GDDR6
Memory (RAM)	16 GB DDR4 3200 MHz
Storage	512 GB SSD

b. Analisis Perangkat Lunak

Spesifikasi perangkat lunak yang digunakan dalam penelitian ini disajikan pada Tabel 3.9

Tabel 3.9 Spesifikasi Perangkat Lunak

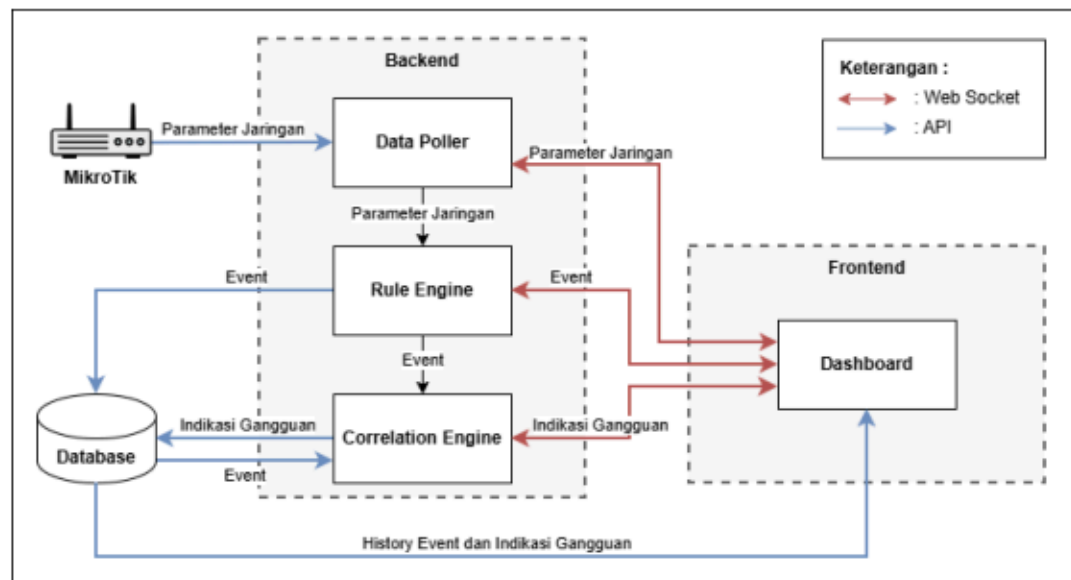
Komponen	Spesifikasi
Sistem Operasi	PopOS 22.04 LTS
Bahasa Pemrograman	JavaScript (Node.js)
Framework Backend	Express.js
Framework Frontend	React.js
Database	MySQL
ORM	Sequelize

3.2.2 Perancangan Desain

Tahap perancangan desain berfokus pada perencanaan desain sistem monitoring yang mampu menampilkan parameter kesehatan jaringan dan analisis indikasi masalah berbasis event. Perancangan desain sistem mencakup perancangan arsitektur sistem dan perancangan wireframe antarmuka sistem.

1. Perancangan Arsitektur Sistem

Perancangan arsitektur sistem merupakan tahapan dimana struktur utama dari sistem dibentuk, mencakup identifikasi komponen inti beserta interaksi antar komponen tersebut. Gambar 3.6 menunjukkan arsitektur sistem monitoring yang dirancang dalam penelitian ini.

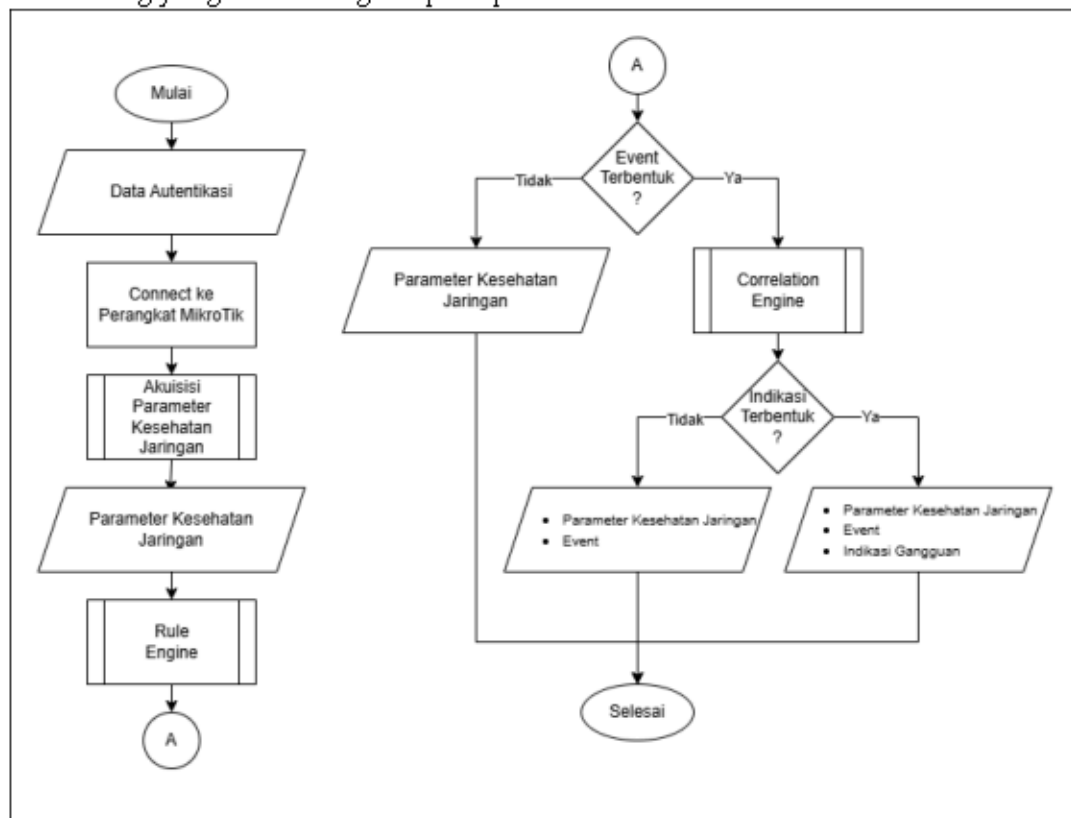


Gambar 3.6 Arsitektur Sistem Monitoring

Gambar 3.6 menunjukkan proses monitoring dimulai dari pengambilan parameter jaringan dari perangkat MikroTik melalui *data poller* secara periodik, kemudian parameter tersebut ditampilkan pada dashboard melalui *web socket* sekaligus di analisis menggunakan *rule engine* untuk mendeteksi dan membentuk event berdasarkan threshold dan durasi yang telah di tetapkan. Event yang terbentuk kemudian ditampilkan pada dashboard sekaligus di analisis menggunakan *correlation engine* untuk menentukan keterkaitan event berdasarkan waktu kemunculan dan keterkaitan parameternya.

2. Perancangan Flowchart Sistem

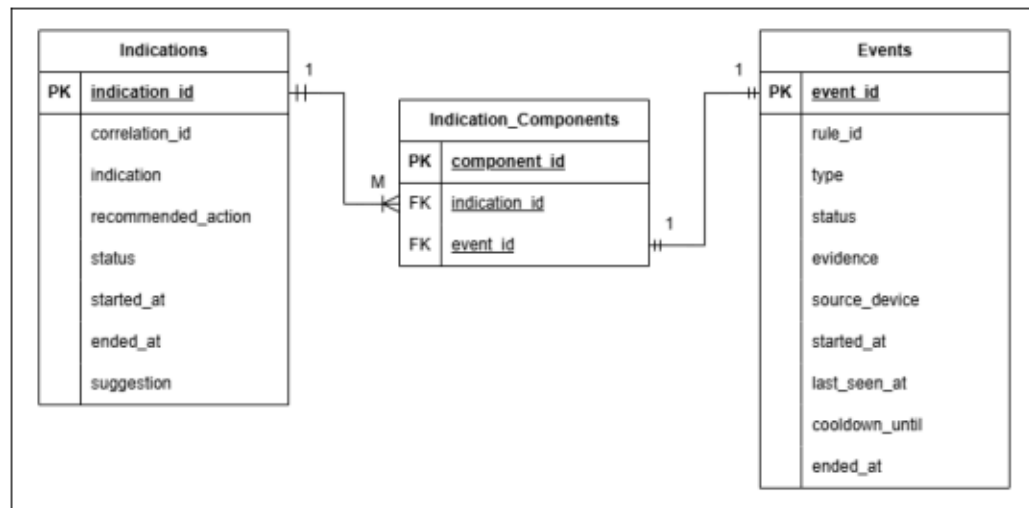
Perancangan *flowchart* merupakan tahap perancangan alur secara keseluruhan sistem yang akan dikembangkan. Gambar 3.7 menunjukkan flowchart sistem monitoring yang dikembangkan pada penelitian ini.



Gambar 3.7 Flowchart Sistem Monitoring

3. Perancangan Basis Data Sistem

Perancangan basis data merupakan tahap penyusunan model tabel, relasi antar tabel, dan struktur database yang digunakan dalam sistem yang dikembangkan. Gambar 3.8 menunjukkan Entity Relationship Diagram (ERD) dari sistem monitoring yang dikembangkan.

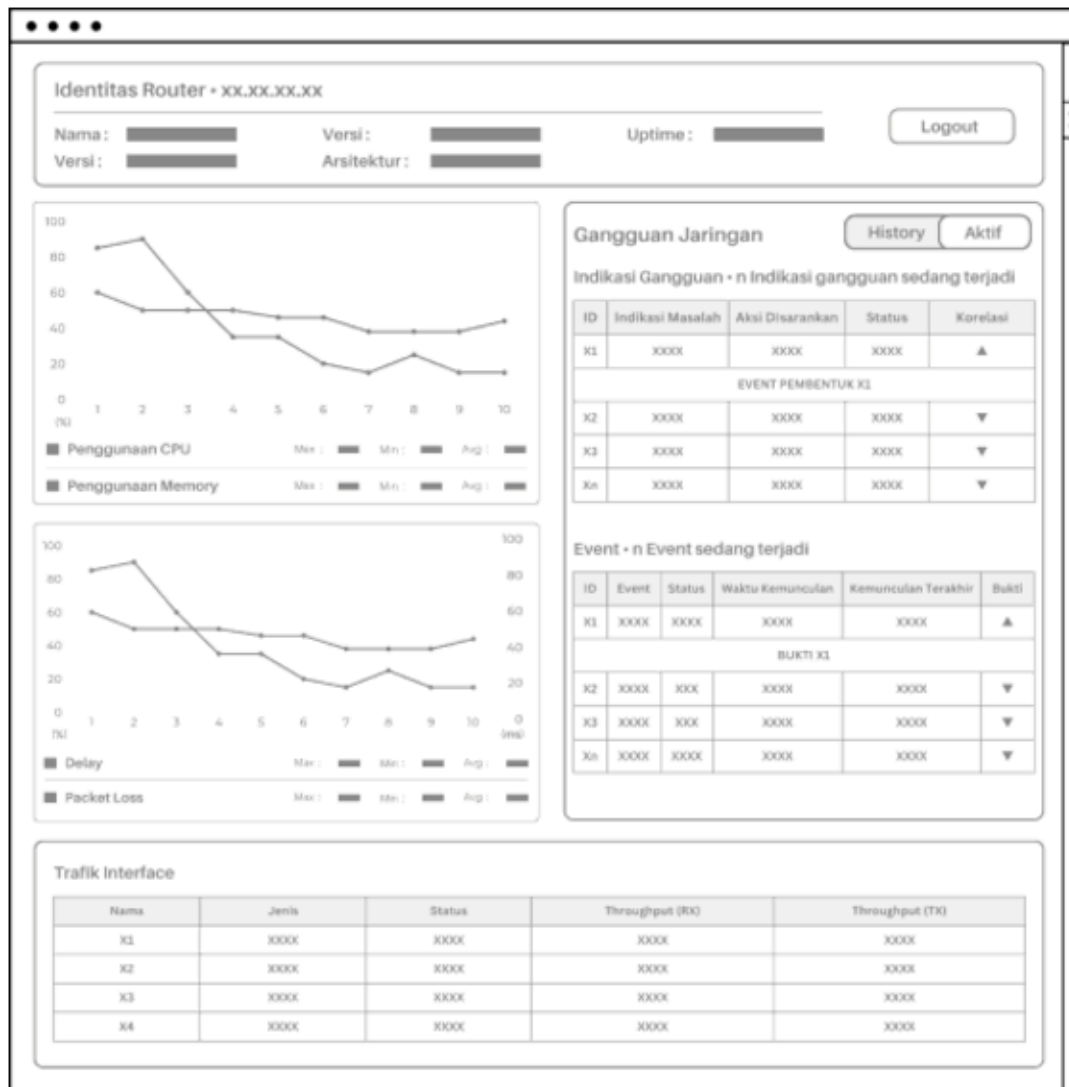


Gambar 3.8 Entity Relationship Diagram (ERD) Sistem Monitoring

4. Perancangan Wireframe Sistem

Perancangan wireframe merupakan tahapan pembuatan desain antarmuka serta alur interaksi pengguna dengan sistem. Gambar 3.9 dan Gambar 3.10 menunjukkan desain antarmuka sistem monitoring yang dikembangkan dalam bentuk wireframe.

Gambar 3.9 Wireframe Login Page Sistem Monitoring



Gambar 3.10 Wireframe Dashboard Sistem Monitoring

3.2.3 Implementasi

Tahap implementasi sistem merupakan tahap akhir dalam pengembangan sistem dalam penelitian ini sebelum kemudian akan dilakukan pengujian. Sistem monitoring dikembangkan dalam bentuk website dengan bahasa pemrograman *JavaScript* dan menggunakan menggunakan arsitektur *client-server* dengan sisi *backend* menggunakan *framework express.js* dan sisi *frontend* menggunakan *framework react.js*. Tahapan coding dibagi menjadi beberapa modul terpisah antara lain:

1. Modul poller, untuk mengambil parameter kesehatan jaringan dari perangkat MikroTik,
2. Modul rule engine, untuk melakukan pembentukan event beserta life cyclenya dengan pendekatan rule-based,
3. Modul correlation engine, untuk menganalisis keterkaitan antar event berdasarkan waktu kejadian dan keterkaitan antar parameter pembentuknya,
4. dan Dashboard monitoring, untuk menampilkan grafik kesehatan jaringan serta event atau indikasi gangguan yang sedang aktif atau historis.

3.2.4 Pengujian

Tahap pengujian sistem dilakukan untuk memastikan bahwa seluruh sistem pada sistem yang dikembangkan dapat berjalan sesuai kebutuhan fungsional yang telah ditentukan pada tahap analisis kebutuhan sebelumnya. Pengujian dilakukan dengan menggunakan metode blackbox testing. Tabel 3.10 menunjukkan tahapan blackbox testing yang dilakukan dalam penelitian ini.

Tabel 3.10 Skenario Uji Sistem Monitoring

No.	Fitur	Skenario Pengujian	Hasil yang Diharapkan
1.	Autentikasi dan koneksi perangkat	Login dengan data yang valid.	Autentikasi berhasil dan sistem terhubung dengan perangkat MikroTik.
		Login dengan data yang tidak valid.	Login gagal dengan pesan kesalahan kredensial.
		Data login valid tetapi perangkat MikroTik tidak dapat dijangkau.	Login gagal dengan pesan koneksi gagal.
2.	Pengambilan parameter jaringan	Sistem menjalankan proses polling parameter kesehatan jaringan.	Parameter kesehatan jaringan diterima dan tampil di dashboard
3.	Pembentukan event gangguan	Kondisi salah satu parameter memenuhi rule.	Event beserta evidence terbentuk dan berhasil ditampilkan.
4.	Pembentukan indikasi gangguan	Beberapa event yang berkaitan muncul dalam window korelasi	Indikasi beserta event pembentuknya terbentuk dan berhasil ditampilkan.
5.	Penyimpanan hasil analisis	Event atau indikasi gangguan terbentuk	Data event, indikasi, dan relasi keduanya tersimpan di database

BAB IV

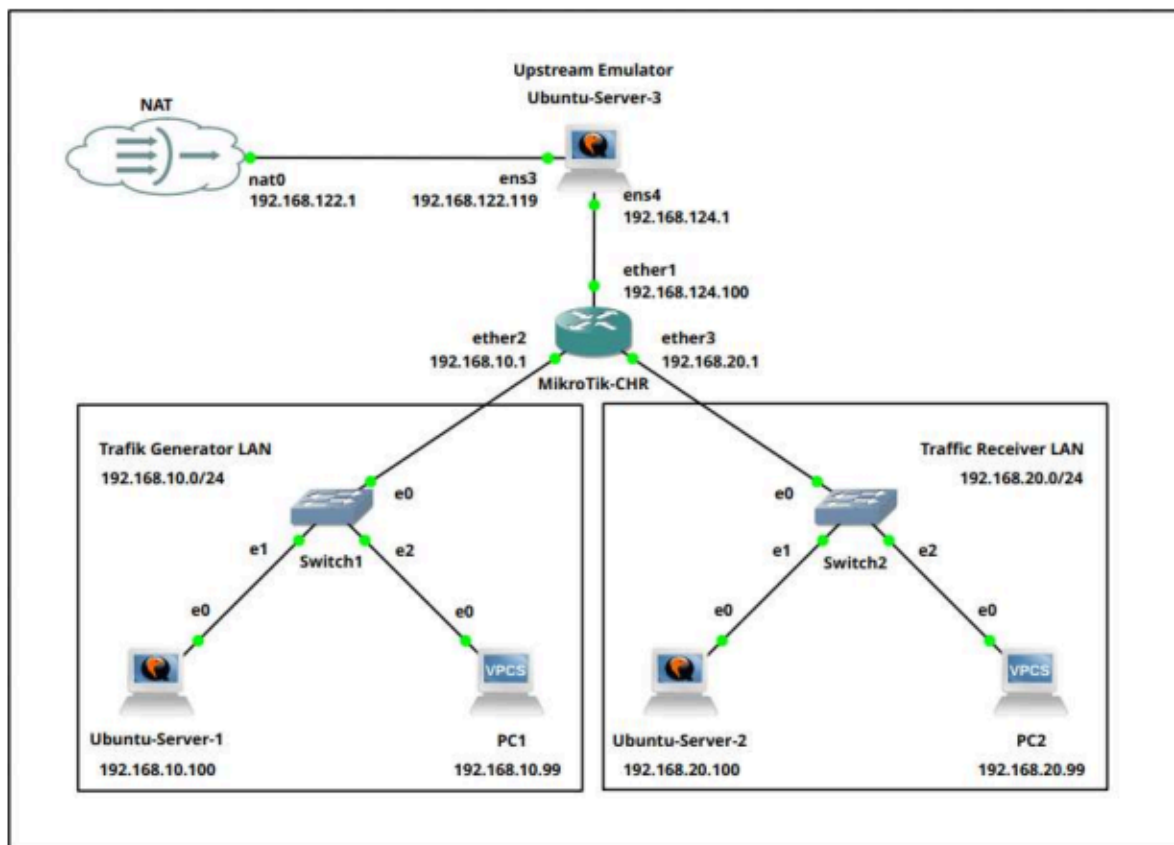
HASIL PENGUJIAN DAN PEMBAHASAN

4.1 Implementasi

Bab ini menjelaskan implementasi sistem monitoring kesehatan jaringan berdasarkan perancangan pada Bab III. Proses implementasi dilakukan secara bertahap, meliputi pembangunan lingkungan uji, pengembangan mekanisme pengambilan dan pengolahan data, serta penerapan metode analisis gangguan jaringan menggunakan *rule-based detection* untuk menghasilkan *event* dan *event correlation* untuk mengaitkan *event* yang terjadi sehingga diperoleh indikasi awal gangguan jaringan beserta *event life cycle* secara terstruktur, serta penerapan arsitektur sistem.

4.1.1 Implementasi Lingkungan Uji

Implementasi lingkungan uji dilakukan dengan membangun topologi jaringan pada GNS3 sesuai dengan rancangan pada Bab III. Tahap ini mencakup penyusunan topologi serta konfigurasi pada setiap *node* agar dapat saling terhubung dan menjalankan fungsinya masing-masing. Topologi jaringan yang telah dibangun ditunjukkan pada Gambar 4.1.



Gambar 4.1 Implementasi Topologi Lingkungan Uji

Setelah topologi selesai dibangun, konfigurasi dilakukan pada setiap node agar dapat saling terhubung dan menjalankan fungsinya. Node *upstream emulator* dikonfigurasi untuk menerima konektivitas dari NAT dan meneruskannya ke *MikroTik CHR* sebagai jalur utama

komunikasi. Agar layanan pada CHR dapat diakses oleh host, CHR perlu berada dalam jangkauan jaringan NAT (192.168.122.0/24). Namun, penggunaan subnet yang sama secara langsung dapat menimbulkan *neighbor conflict* pada jalur balik paket karena *upstream emulator* memiliki dua antarmuka dengan gateway yang identik. Untuk mengatasinya, digunakan dua subnet berbeda, yaitu 192.168.122.0/24 pada sisi NAT dan 192.168.124.0/24 pada koneksi ke CHR, sehingga jalur *routing* menjadi lebih jelas dan tidak ambigu. Konsekuensinya, CHR tidak berada langsung dalam subnet NAT, sehingga diterapkan konfigurasi *port forwarding* dari port 8080 pada *upstream emulator* ke port 80 pada CHR agar layanan tetap dapat diakses melalui NAT. Selain itu, mekanisme *traffic control* menggunakan *netem* diterapkan untuk mensimulasikan kondisi jaringan seperti *delay* dan *packet loss*. Hasil konfigurasi ditunjukkan pada Gambar 4.2 dan Gambar 4.3.



```
File Edit View          tsu@pop-os: ~          + - ↗ ✕

tsu@pop-os:~$ ping 192.168.122.119 -c 5 && echo ""
PING 192.168.122.119 (192.168.122.119) 56(84) bytes of data.
 64 bytes from 192.168.122.119: icmp_seq=1 ttl=64 time=0.297 ms
 64 bytes from 192.168.122.119: icmp_seq=2 ttl=64 time=0.330 ms
 64 bytes from 192.168.122.119: icmp_seq=3 ttl=64 time=0.286 ms
 64 bytes from 192.168.122.119: icmp_seq=4 ttl=64 time=0.460 ms
 64 bytes from 192.168.122.119: icmp_seq=5 ttl=64 time=0.315 ms

--- 192.168.122.119 ping statistics ---
 5 packets transmitted, 5 received, 0% packet loss, time 4122ms
 rtt min/avg/max/mdev = 0.286/0.337/0.460/0.063 ms

tsu@pop-os:~$ curl -s -u admin:admin http://192.168.122.119:8080/rest/system/identity | jq
{
  "name": "MikroTik"
}
tsu@pop-os:~$
```

Host berhasil berkomunikasi dengan Upstream Emulator

Host berhasil mengakses layanan Mikrotik CHR melalui port forwarding Upstream Emulator

Gambar 4.2 Konektivitas Host ke Upstream Emulator dan MikroTik CHR

The image shows two terminal windows. The top window is titled 'MikroTik-CHR-1' and displays the output of a DHCP configuration and a ping test. The bottom window is titled 'Ubuntu-Server-3' and shows the execution of a 'tc netem' command to inject network delays and packet loss.

```

MikroTik-CHR-1
File Edit View Search Terminal Help
Columns: NAME, INTERFACE, ADDRESS-POOL, LEASE-TIME
# NAME      INTERFACE ADDRESS-POOL  LEASE-TIME
0 dhcp-generator ether2    pool-generator 30m
1 dhcp-receiver ether3    pool-receiver  30m
[admin@MikroTik] > ping 192.168.122.1 count=5
SEQ HOST      SIZE TTL TIME          STATUS
0 192.168.122.1 56 63 652us        OK
1 192.168.122.1 56 63 1ms332us    OK
2 192.168.122.1 56 63 1ms131us    OK
3 192.168.122.1 56 63 1ms8us     OK
4 192.168.122.1 56 63 1ms319us   OK
sent=5 received=5 packet-loss=0% min-rtt=652us avg-rtt=1ms88us max-rtt=1ms332us
Kondisi awal (Baseline/ Tanpa gangguan)

[admin@MikroTik] > ping 192.168.122.1 count=5
SEQ HOST      SIZE TTL TIME          STATUS
0 192.168.122.1 56 63 201ms87us   OK
1 192.168.122.1 56 63 201ms374us  timeout
2 192.168.122.1 56 63 201ms374us  timeout
3 192.168.122.1 56 63 201ms221us  timeout
4 192.168.122.1 56 63 201ms221us  timeout
sent=5 received=3 packet-loss=40% min-rtt=201ms87us avg-rtt=201ms227us max-rtt=201ms374us
Kondisi setelah injeksi gangguan jaringan

[admin@MikroTik] >

Ubuntu-Server-3
File Edit View Search Terminal Help
ubuntu@ubuntu-cloud:~$ sudo tc qdisc add dev ens4 root netem delay 200ms loss 50%
ubuntu@ubuntu-cloud:~$
Perintah injeksi gangguan dengan tc netem

```

Gambar 4.3 Implementasi Network Emulator dengan Netem pada Upstream Emulator

Node *MikroTik CHR* dikonfigurasi untuk menerima koneksi dari *upstream emulator*, mendistribusikan alamat IP ke dua segmen jaringan *downstream* melalui layanan *DHCP server*, serta mengatur *routing* antar segmen. Selain itu, ditambahkan *firewall filter* dengan aksi *logging* pada trafik TCP, UDP, dan ICMP di *chain forward* untuk merekam trafik uji sekaligus meningkatkan beban CPU saat terjadi *flood* dari *traffic generator*. Dengan demikian, seluruh trafik uji diproses dan dicatat oleh CHR sebelum diteruskan, sehingga memberikan dampak langsung pada penggunaan sumber daya. Hasil konfigurasi ditunjukkan pada Gambar 4.4 dan Gambar 4.5.

```

MikroTik-CHR-1
File Edit View Search Terminal Help
[admin@MikroTik] > /ip/address/print
Columns: ADDRESS, NETWORK, INTERFACE
# ADDRESS NETWORK INTERFACE
0 192.168.124.100/24 192.168.124.0 ether1
1 192.168.10.1/24 192.168.10.0 ether2
2 192.168.20.1/24 192.168.20.0 ether3
[admin@MikroTik] > /ip/route/print
Flags: D - DYNAMIC; A - ACTIVE; c - CONNECT, s - STATIC
Columns: DST-ADDRESS, GATEWAY, ROUTING-TABLE, DISTANCE
# DST-ADDRESS GATEWAY ROUTING-TABLE DISTANCE
0 As 0.0.0.0/0 192.168.124.1 main 1
DAC 192.168.10.0/24 ether2 main 0
DAC 192.168.20.0/24 ether3 main 0
DAC 192.168.124.0/24 ether1 main 0
[admin@MikroTik] > /ip/pool/print
Columns: NAME, RANGES, TOTAL, USED, AVAILABLE
# NAME RANGES TOTAL USED AVAILABLE
0 pool-generator 192.168.10.10-192.168.10.100 91 2 89
1 pool-receiver 192.168.20.10-192.168.20.100 91 2 89
[admin@MikroTik] > /ip/dhcp-server/print
Columns: NAME, INTERFACE, ADDRESS-POOL, LEASE-TIME
# NAME INTERFACE ADDRESS-POOL LEASE-TIME
0 dhcp-generator ether2 pool-generator 30m
1 dhcp-receiver ether3 pool-receiver 30m
[admin@MikroTik] >

```

Gambar 4.4 Konfigurasi IP dan Distribusi DHCP pada MikroTik CHR

```

MikroTik-CHR
File Edit View Search Terminal Help
[admin@MikroTik] > /ip/firewall/filter/print
Flags: X - disabled, I - invalid; D - dynamic
0 chain=forward action=log protocol=tcp log-prefix="TCP-FW"
1 chain=forward action=log protocol=udp log-prefix="UDP-FW"
2 chain=forward action=log protocol=icmp log-prefix="ICMP-FW"
[admin@MikroTik] >

```

Gambar 4.5 Konfigurasi Firewall Filter pada MikroTik CHR

Node Ubuntu pada sisi downstream dikonfigurasi untuk memperoleh alamat IP secara otomatis dari DHCP server pada CHR. Hasilnya, kedua node (trafik generator dan penerima trafik) dapat saling berkomunikasi serta mengakses jaringan eksternal. Kondisi ini ditunjukkan pada Gambar 4.6.



Gambar 4.6 Konektivitas Antar Node Downstream dan Akses ke Jaringan Eksternal

4.1.2 Implementasi Pengambilan dan Pengolahan Data

Implementasi pengambilan dan pengolahan data dilakukan dengan membangun mekanisme pengambilan dan pengolahan data dari MikroTik melalui RouterOS API sesuai dengan rancangan yang telah dijelaskan pada Bab III.

Mekanisme tersebut dibagi ke dalam dua komponen utama, yaitu modul pengambilan data dan modul pengolahan data. Modul pengambilan data bertanggung jawab untuk mengakses RouterOS API dan mengambil data mentah dari perangkat MikroTik secara periodik, sedangkan modul pengolahan data bertugas untuk mengolah data mentah tersebut menjadi informasi yang lebih terstruktur dan siap digunakan oleh sistem.

1. Mekanisme Pengambilan Data

Pengambilan data diawali dengan membangun mekanisme *HTTP client* untuk terhubung ke *RouterOS API* pada perangkat MikroTik. Koneksi ini diimplementasikan menggunakan *library Axios* dalam bentuk *class* MikroTikClient. Konfigurasi mencakup alamat host, autentikasi, dan pengaturan *timeout*. Implementasi tersebut ditunjukkan pada Source Code 1.

Source Code 1. Mekanisme koneksi ke RouterOS API

```
// MikroTik.js
export class MikroTikClient {
  constructor() {
    this.cfg = null; // menyimpan konfigurasi koneksi aktif
    this.http = null; // instance HTTP client untuk akses API MikroTik
  }

  configure(cfg) {
    this.cfg = { ...cfg }; // inisialisasi konfigurasi koneksi
    const baseUrl = `http://${(cfg.host):8080}/rest`;

    this.http = axios.create({
      baseUrl,
      timeout: 15000, // batas waktu request untuk mencegah hang
      auth: { username: cfg.username, password: cfg.password } // autentikasi
    });

    return true; // menandakan koneksi berhasil
  }

  /* seterusnya... */
}
```

Setelah mekanisme koneksi ke perangkat MikroTik berhasil diimplementasikan, langkah selanjutnya adalah melakukan pengambilan data parameter kesehatan jaringan. Data yang diambil pada tahap ini terdiri dari dua jenis, yaitu data statis dan data dinamis. Data statis berupa identitas perangkat, sedangkan data dinamis meliputi performa perangkat, kualitas layanan jaringan, serta konektivitas *interface*, sebagaimana telah dirancang pada tahap metodologi.

Pengambilan data statis dilakukan satu kali saat koneksi ke MikroTik berhasil, karena nilainya tidak berubah selama sistem berjalan dan hanya digunakan sebagai identifikasi perangkat. Data ini diperoleh melalui *endpoint /system/identity* dan */system/resource*, dengan implementasi ditunjukkan pada Source Code 2.

Source Code 2. Pengambilan data identitas perangkat MikroTik

```
// mikrotik.js
async getIdentity() {

  // mengambil data identitas dan resource perangkat dari MikroTik
  const ( data: identity ) = await this.http.get("/system/identity");
  const ( data: resource ) = await this.http.get("/system/resource");

  // mengembalikan field data identitas dan resource yang dibutuhkan
  return {
    name: identity?.name,
    version: resource?.version,
    platform: resource?.platform,
    cpu: resource?.cpu,
    cpu_count: Number(resource?.["cpu-count"]),
    architecture_name: resource?.["architecture-name"],
    total_memory: Number(resource?.["total-memory"]),
    board_name: resource?.["board-name"],
  };
}
```

Pengambilan data dinamis diimplementasikan menggunakan mekanisme yang serupa, yaitu melalui pemanggilan endpoint RouterOS API sesuai dengan parameter yang dibutuhkan. Proses ini dijalankan secara periodik menggunakan mekanisme polling dengan interval yang telah ditentukan.

Data performa perangkat diperoleh melalui *endpoint /system/resource*, data konektivitas *interface* melalui *endpoint /interface*, dan data kualitas layanan jaringan diperoleh melalui mekanisme ping menggunakan *endpoint /tool/ping*. Implementasi mekanisme tersebut ditunjukkan pada Source Code 3, Source Code 4, dan Source Code 5.

Source Code 3. Pengambilan data resource perangkat MikroTik

```
// mikrotik.js
async getResource() {

  // mengambil data resource perangkat dari MikroTik
  const ( data ) = await this.http.get("/system/resource");

  // mengembalikan field data resource yang dibutuhkan
  return {
    "cpu-load": data?.["cpu-load"],
    "total-memory": data?.["total-memory"],
    "free-memory": data?.["free-memory"],
    uptime: data?.uptime
  };
}
```

Source Code 4. Pengambilan data kualitas layanan jaringan

```
// mikrotik.js
async ping(target, count = 10, intervalMs = 450) {

  // membentuk request body untuk pengujian ping
  const body = { address: target, count, interval: `${intervalMs}ms` };

  // mengambil data hasil ping dari RouterOS
  const { data } = await this.http.post("/tool/ping", body);

  // mengembalikan field data ping yang dibutuhkan
  return data.map(r => ({
    sent: r.sent,
    received: r.received,
    time: r.time
  }));
}
```

Source Code 5. Pengambilan data konektivitas interface

```
// mikrotik.js
async getInterfaces() {

  // mengambil data interface dari perangkat Mikrotik
  const { data } = await this.http.get("/interface");

  // mengembalikan field data interface yang dibutuhkan
  return data.map(iface => ({
    id: iface?["id"],
    name: iface?.name,
    type: iface?.type,
    running: iface?.running,
    disabled: iface?.disabled
  }));
}
```

Mekanisme pengambilan data mentah yang telah diimplementasikan selanjutnya diintegrasikan ke dalam *poller* untuk menjalankan proses pengambilan data secara periodik. *Poller* mengatur siklus eksekusi secara sekuensial, di mana setiap fungsi pengambilan sekaligus pengolahan data dipanggil dalam satu rangkaian proses. Interval pengambilan dikendalikan secara adaptif, yaitu siklus berikutnya dijalankan setelah seluruh proses selesai dengan jeda minimal 5 detik sesuai rancangan, sehingga tidak terjadi *overlap* antarproses. Implementasi mekanisme tersebut ditunjukkan pada Source Code 6.

Source Code 6. Inisialisasi mekanisme poller

```
// poller.js
export class Poller {
  /* inisialisasi poller.. */
  async stop() {
    this.isRunning = false;
    if (this.loopPromise) await this.loopPromise;
    this.loopPromise = null;
    // .. menghentikan siklus polling dan mereset state internal
  }

  start() {
    this.stop().then(() => {
      this.isRunning = true;
      this.loopPromise = this.runLoop(); // memulai siklus polling
    });
  }

  async runLoop() {
```

Source Code 6. Inisialisasi mekanisme poller

```
while (this.isRunning) {
  const start = Date.now();

  // menjalankan pengambilan dan pengolahan data secara sekuensial
  await this.pollResource();
  await this.pollPing();
  await this.pollInterfaces();

  // menghitung sisa waktu untuk memenuhi interval minimal
  const elapsed = Date.now() - start;
  const wait = Math.max(0, CONFIG.pollingIntervalMs - elapsed);

  // memberikan jeda sebelum siklus berikutnya untuk mencegah overlap
  if (wait > 0 && this.isRunning)
    await new Promise(r => setTimeout(r, wait));
}
}
```

2. Mekanisme Pengolahan Data

Pengolahan data dilakukan setelah data mentah diperoleh dari perangkat MikroTik melalui *poller*. Pada implementasinya, *poller* tidak hanya mengambil data mentah, tetapi juga langsung mengolahnya sehingga data yang dihasilkan siap digunakan untuk analisis selanjutnya. Mekanisme pengolahan data diawali dengan restrukturisasi dan normalisasi data, perhitungan nilai turunan pada parameter tertentu, serta penambahan *timestamp*. Implementasi mekanisme tersebut ditunjukkan pada Source Code 7, Source Code 8, dan Source Code 9.

Source Code 7. Pengolahan data resource

```
// poller.js
async pollResource() {
  // request data resource melalui fungsi pada class MikroTikClient
  const raw = await this.mikrotik.getResource();

  // parsePercent: mengonversi nilai persen string menjadi numerik
  // |contoh: "10%" menjadi 10|
  const cpuLoad = parsePercent(raw?.["cpu-load"]);

  // parseMiB: mengonversi nilai memori string ke satuan MiB
  // |contoh: "1GiB" menjadi 1024|
  const totalMem = parseMiB(raw?.["total-memory"]);
  const freeMem = parseMiB(raw?.["free-memory"]);

  // perhitungan nilai turunan memori
  const usedMem = Math.max(0, totalMem - freeMem);
  const freeMemPct = totalMem ? (freeMem / totalMem) * 100 : 0;
  const usedMemPct = totalMem ? (usedMem / totalMem) * 100 : 0;

  // penambahan timestamp sebagai penanda waktu data
  const timestamp = Date.now();

  /* seterusnya.. */
}
```

Source Code 8. Pengolahan data kualitas layanan jaringan

```
// poller.js
async pollPing() {
  // request ping melalui fungsi pada class MikroTikClient
  const raw = await this.mikrotik.ping(target, 10, 450);
}
```

Source Code 8. Pengolahan data kualitas layanan jaringan

```
// mengambil jumlah paket terkirim dan diterima dalam satu pengujian ping
const sentVals = raw.map|r => Number|r?.sent|}
  .filter|Number.isFinite|;
const receivedVals = raw.map|r => Number|r?.received|}
  .filter|Number.isFinite|;
const packetSent = sentVals.length ? Math.max|...sentVals| : 0;
const packetReceived = receivedVals.length ? Math.max|...receivedVals| :
0;

// parseTimeMs: mengonversi waktu ping menjadi milidetik
// |contoh: "23ms45us" menjadi 23.45 ms|
const times = raw.map|r => r?.time|
  .filter|t => t != null|;
const delayMsArray = times.map|t => parseTimeMs|t|}
  .filter|ms => ms > 0|;

// menghitung rata-rata delay dalam satu pengujian ping
const avgDelayMs = delayMsArray.length
  ? delayMsArray.reduce||a, b| => a + b, 0| / delayMsArray.length
  : 0;

// menambahkan timestamp sebagai penanda waktu data
const timestamp = Date.now|};

/* seterusnya... */
}
```

Source Code 9. Pengolahan data konektivitas interface

```
//poller.js
async pollInterfaces| | {
  // request data interfaces melalui fungsi pada class MikroTikClient
  const rawIfaces = await this.mikrotik.getInterfaces|};
  // normalisasi data interface dan penentuan status operasional
  const processed = rawIfaces.map|iFace => {

    // memastikan nilai boolean dari field running dan disabled
    const running = iFace?.running === true || iFace?.running === "true";
    const disabled = iFace?.disabled === true || iFace?.disabled === "true";

    let status = "down"; // default status interface
    if |disabled| status = "disabled"; // jika interface dinonaktifkan
    else if |running| status = "up"; // jika interface berjalan

    return {
      name: iFace?.name,
      type: iFace?.type,
      status,
      disabled
    };
  }|};

/* seterusnya... */
}
```

Data hasil pengolahan selanjutnya disimpan ke dalam *sliding window* untuk mempertahankan riwayat data dalam rentang waktu tertentu. Mekanisme ini diawali dengan inisialisasi struktur *sliding window*, kemudian dilanjutkan dengan proses memasukkan data hasil pengolahan ke dalam window sesuai parameter masing-masing. Implementasi mekanisme tersebut ditunjukkan pada Source Code 10.

Source Code 10. Pembentukan dan penambahan data pada sliding window

```
// poller.js
class Poller {
  constructor() {
    // inisialisasi sliding window untuk tiap parameter
    this.windows = { cpu: [], memory: [], delay: [], packetLoss: [] };
  }

  async pollResource() {
    /* pengambilan data dan pengolahan.. */
    const timestamp = Date.now();
    // memasukkan data CPU ke window
    // function push akan menambah data baru
    // dan membuang data terlama jika kapasitas penuh
    this.windows.cpu.push({ value: cpuLoad, timestamp });
    // memasukkan data memori ke window (menyimpan persentase free dan used)
    this.windows.memory.push({
      free: freeMemPct,
      used: usedMemPct,
      timestamp
    });
  }

  async pollPing() {
    /* pengambilan data dan pengolahan.. */
    const timestamp = Date.now();
    // memasukkan data delay ke window
    this.windows.delay.push({ value: avgDelayMs, timestamp });
    // memasukkan data packet loss ke window
    this.windows.packetLoss.push({
      sent: packetSent,
      received: packetReceived, timestamp
    });
  }
}
```

Data dalam *sliding window* selanjutnya diolah untuk memperoleh nilai statistik berupa rata-rata, minimum, dan maksimum pada setiap parameter. Proses ini dilakukan dengan mengekstraksi nilai numerik dari *window*, kemudian dilakukan agregasi sesuai kebutuhan. Perhitungan *packet loss* dilakukan dengan menjumlahkan total paket yang dikirim dan diterima, kemudian dihitung selisihnya sebagai jumlah paket yang hilang dan dikonversi menjadi persentase. Hasil perhitungan tersebut digunakan sebagai dasar dalam proses *rule-based detection*, *event correlation*, serta visualisasi data. Implementasinya ditunjukkan pada Source Code 11.

Source Code 11. Pengolahan Statistik Data pada Sliding Window

```
// utils.js

// perhitungan statistik window (rata-rata, minimum, maksimum)
export const windowStats = (window, key) => {
  const values = extractNumericValues(window, key); // mengambil nilai
  numerik dari window

  const avg = calculateAverage(values); // menghitung nilai rata-rata
  const min = calculateMinimum(values); // menghitung nilai minimum
  const max = calculateMaximum(values); // menghitung nilai maksimum

  return { count: values.length, avg, min, max };
};

// perhitungan statistik packet loss berbasis agregasi paket
```

Source Code 11. Pengolahan Statistik Data pada Sliding Window

```
export const windowStatsPacketLoss = (window) => {  
  const totalSent = sumValues(window, "packetSent"); // total paket dikirim  
  const totalReceived = sumValues(window, "packetReceive"); // total paket diterima  
  
  const totalLoss = totalSent - totalReceived; // total paket hilang  
  const lossPct = calculatePercentage(totalLoss, totalSent); // konversi ke persen  
  
  return ( totalSent, totalReceived, totalLoss, lossPct );  
};
```

4.1.3 Implementasi Rule-Based Detection

Implementasi *rule-based detection* dilakukan dengan membangun *rule engine* yang bertugas mengevaluasi setiap data hasil monitoring untuk membentuk *event* berdasarkan rule yang telah dirancang. Secara umum, rule bekerja dengan membandingkan nilai parameter terhadap *threshold* serta memastikan jumlah data dalam *window* telah memenuhi durasi minimum. Namun, pada kondisi tertentu seperti parameter interface, evaluasi dilakukan berdasarkan perubahan status (*state transition*) dari kondisi sebelumnya. Jika kondisi yang ditentukan pada masing-masing rule terpenuhi, maka *event* akan dibentuk atau diperbarui. Implementasi masing-masing rule pada *rule engine* ditunjukkan pada Source Code 12 sampai Source Code 15.

Source Code 12. Implementasi rule R1 (interface) kedalam rule engine

```
// ruleEngine.js  
async onInterfaceData(routerName, routerIp, interfaces) {  
  const now = new Date();  
  const prev = this.previousInterfaceStatus.get(routerIp); // status sebelumnya  
  const curr = new Map();  
  
  for (const iface of interfaces) {  
    const isUp = iface.status === 'up'; // kondisi saat ini  
    curr.set(iface.name, isUp);  
    const prevState = prev?.get(iface.name);  
  
    // jika sebelumnya up dan sekarang down - event terbentuk  
    if (prevState === true && isUp === false) {  
      await this.models.Event.create({ /* data event */ });  
  
    // jika sebelumnya down dan sekarang up - event diakhiri  
    if (prevState === false && isUp === true) {  
      await this.models.Event.update({  
        ( status:'ended', endedAt:now ),  
        ( where:( routerIp, type:RULES.R1.type, status:'active' ) )  
      });  
    }  
  }  
}  
  
// simpan kondisi terbaru sebagai pembanding berikutnya  
this.previousInterfaceStatus.set(routerIp, curr);  
}
```

Source Code 13. Implementasi rule R2 (delay) kedalam rule engine

```
// ruleEngine.js
async onLatencyData(routerName, routerIp, data) {
  const ( timestamp ) = data;
  const type = RULES.R2.type;
  let event = /* ambil event aktif/cooldown */;
  const now = new Date(timestamp);

  // ambil data window delay dan hitung rata-rata
  const delayData = this.poller?.getWindowData('delay', timestamp - 60000) || [];
  const avgPrev = delayData.length
    ? delayData.reduce((s, d) => s + d.avgMs, 0) / delayData.length
    : 0;

  // cek apakah threshold dan jumlah data window terpenuhi
  const active =
    delayData.length >= LIB.windows.ping.max
    && avgPrev >= LIB.thresholds.latencyAvgMs;

  if (active) {
    // jika threshold dan durasi terpenuhi - event terbentuk atau diperbarui
    if (!event || event.status === 'ended') {
      // belum ada event - buat event baru
      event = await this.models.Event.create({ /* data event */ });
    } else {
      // sudah ada event
      const updates = { lastSeenAt: now };

      // jika sebelumnya cooldown - aktif kembali
      if (event.status === 'cooldown') {
        updates.status = 'active';
        updates.cooldownUntil = null;
      }

      await event.update(updates);
    }
  } else {
    // jika threshold tidak terpenuhi
    if (event && (event.status === 'active' || event.status === 'cooldown')) {
      if (event.status === 'active') {
        // sebelumnya aktif, sekarang kondisi hilang - masuk cooldown
        await event.update({
          status: 'cooldown',
          cooldownUntil: new Date(now.getTime() + LIB.cooldownMs),
          lastSeenAt: now
        });
      } else if (event.status === 'cooldown' && now >= event.cooldownUntil) {
        // sudah cooldown dan waktunya habis - event diakhiri
        await event.update({ status: 'ended', endedAt: now });
      }
    }
  }
}
```

Source Code 14. Implementasi rule R3 (packet loss) kedalam rule engine

```
// ruleEngine.js
async onPacketLossData(routerName, routerIp, data) {
  const { timestamp, windowStats } = data;
  const type = RULES.R3.type;
  let event = /* ambil event aktif/cooldown */;
  const now = new Date(timestamp);

  const avgPrev = windowStats?.avg ?? 0;
  const windowCount = windowStats?.count ?? 0;

  // cek threshold packet loss
  const active =
    windowCount >= LIB.windows.ping.max
    && avgPrev >= LIB.thresholds.packetLossAvgPct;

  if !active {
    // jika threshold dan durasi terpenuhi → event terbentuk
    if !event || event.status==='ended' {
      event = await this.models.Event.create({ /* data event */ });
    } else {
      const updates = { lastSeenAt:now };

      // jika sebelumnya cooldown → aktif kembali
      if !event.status==='cooldown' {
        updates.status='active';
        updates.cooldownUntil=null;
      }

      await event.update(updates);
    }
  } else {
    // jika kondisi tidak terpenuhi
    if !event && !event.status==='active' || event.status==='cooldown' {
      if !event.status==='active' {
        // kondisi kembali normal → masuk cooldown
        await event.update({
          status:'cooldown',
          cooldownUntil:new Date(now.getTime()+LIB.cooldownMs),
          lastSeenAt:now
        });
      } else if !event.status==='cooldown' && now>=event.cooldownUntil {
        // cooldown selesai → event diakhiri
        await event.update({ status:'ended', endedAt:now });
      }
    }
  }
}
```

Source Code 15. Implementasi rule R4 (cpu) kedalam rule engine

```
// ruleEngine.js
async onCpuData(routerName, routerIp, data) {
  const { timestamp } = data;
  const type = RULES.R4.type;
  let event = /* ambil event aktif/cooldown */;
```

Source Code 15. Implementasi rule R4 (cpu) kedalam rule engine

```
const now = new Date(timestamp);

// ambil data CPU dari window
const cpuData = this.poller?.getWindowData('cpu', timestamp - 60000) || [];
const avg = cpuData.length
  ? cpuData.reduce((s,d)=>s+d.loadPct,0)/cpuData.length
  : 0;

// cek threshold CPU
const active =
  cpuData.length >= LIB.windows.cpu.max
  && avg >= LIB.thresholds.cpuAvgPct;

if (active) {
  // jika threshold dan durasi terpenuhi → event terbentuk
  if (!event || event.status==='ended') {
    event = await this.models.Event.create({ /* data event */ });
  } else {
    const updates = { lastSeenAt:now };

    // jika sebelumnya cooldown → aktif kembali
    if (event.status==='cooldown') {
      updates.status='active';
      updates.cooldownUntil=null;
    }

    await event.update(updates);
  }
} else {
  // jika kondisi tidak terpenuhi
  if (event && (event.status==='active' || event.status==='cooldown')) {

    if (event.status==='active') {
      // CPU kembali normal → masuk cooldown
      await event.update({
        status:'cooldown',
        cooldownUntil:new Date(now.getTime()+LIB.cooldownMs),
        lastSeenAt:now
      });
    }

    if (event.status==='cooldown' && now>=event.cooldownUntil) {
      // cooldown selesai → event diakhiri
      await event.update({ status:'ended', endedAt:now });
    }
  }
}
```

Fungsi dari *rule engine* yang telah diimplementasikan kemudian dipanggil pada setiap proses pengambilan data di dalam *poller*. Setiap data yang diperoleh langsung dikirim ke *rule engine* untuk dilakukan evaluasi sesuai rule yang berlaku. Implementasi ini ditunjukkan pada Source Code 16.

Source Code 16. Implementasi rule engine pada poller

```
// poller.js
async pollResource() {
  /* pengambilan, pengolahan, push ke window {CPU & memory} */
  if (this.ruleEngine) {
    // memanggil onCpuData
    await this.ruleEngine.onCpuData()
    /* data CPU {window, statistik window, routerInfo, timestamp} */
  };
}

async pollPing() {
  /* pengambilan, pengolahan, push ke window {delay & packet loss} */
  if (this.ruleEngine) {
    // memanggil onDelayData
    await this.ruleEngine.onDelayData()
    /* data delay {window, statistik window, routerInfo, timestamp} */
  };
  // memanggil onPacketLossData
  await this.ruleEngine.onPacketLossData()
  /* data packet loss {window, statistik window, routerInfo, timestamp} */
  };
}

async pollInterfaces() {
  /* pengambilan, pengolahan data interface */
  if (this.ruleEngine) {
    // memanggil onInterfaceData
    await this.ruleEngine.onInterfaceData()
    /* data interface {status sebelumnya, status saat ini, routerInfo} */
  };
}
}
```

4.1.4 Implementasi Event Lifecycle

Mekanisme *event life cycle* diimplementasikan dalam *rule engine* saat evaluasi kondisi (*rule*). Jika *threshold* dan jumlah data dalam *window* terpenuhi, maka sistem akan membuat event baru apabila belum ada atau event sebelumnya telah berstatus *ended*. Jika event sudah ada dan berstatus *active* atau *cooldown*, maka status dikembalikan menjadi *active*, *lastSeenAt* diperbarui, dan *cooldownUntil* dihapus.

Apabila kondisi tidak memenuhi *threshold*, maka event yang berstatus *active* akan diubah menjadi *cooldown*. Selanjutnya, apabila waktu *cooldownUntil* telah terlewati, status event diubah menjadi *ended*. Mekanisme ini ditunjukkan pada Source Code 14, Source Code 15, dan Source Code 16.

4.1.5 Implementasi Event Correlation

Implementasi *event correlation* dilakukan dengan membangun *correlation engine* untuk mengidentifikasi keterkaitan antar *event* hasil *rule-based detection*. Berbeda dengan *rule* yang hanya mengevaluasi satu parameter, *correlation engine* menggabungkan beberapa event berdasarkan keterkaitan parameter dan kedekatan waktu untuk menghasilkan indikasi

gangguan yang lebih kontekstual. Mekanisme ini diintegrasikan pada *rule engine*, sehingga setiap event yang terbentuk akan langsung memicu proses evaluasi dari *correlation engine*.

Proses evaluasi pada *correlation engine* dimulai dengan mengambil seluruh event berstatus *active* atau *cooldown*, kemudian mengelompokkannya berdasarkan tipe (delay, *packet loss*, dan CPU). Selanjutnya, sistem mengevaluasi apakah kombinasi event tersebut memenuhi pola korelasi dalam rentang waktu tertentu (*correlation window*). Jika terpenuhi, sistem akan membentuk atau memperbarui *indication*. Sebaliknya, jika kondisi tidak terpenuhi atau seluruh event terkait telah berakhir, maka *indication* juga diakhiri. Implementasi dari mekanisme ini ditunjukkan pada Source Code 17.

Source Code 17. Inisialisasi *correlation engine*

```
// correlationEngine.js
async evaluateRouter(routerIp) {
  const activeEvents = /* ambil event aktif/cooldown */;

  // kelompokkan event berdasarkan rule
  const r2Events = activeEvents.filter(e => e.type === RULES.R2.type);
  const r3Events = activeEvents.filter(e => e.type === RULES.R3.type);
  const r4Events = activeEvents.filter(e => e.type === RULES.R4.type);

  // cek keberadaan event dari masing-masing parameter
  const hasR2 = r2Events.length > 0;
  const hasR3 = r3Events.length > 0;
  const hasR4 = r4Events.length > 0;

  // evaluasi pola C2 {delay + loss + cpu}
  let c2Valid = false;
  if (hasR2 && hasR3 && hasR4) {
    // hitung rentang waktu antar event
    const timestamps = [...r2Events, ...r3Events, ...r4Events]
      .map(e => new Date(e.startedAt).getTime());
    const range = Math.max(...timestamps) - Math.min(...timestamps);
    // jika masih ddalam window - pola terpenuhi
    if (range <= CONFIG.correlationWindowMs) {
      c2Valid = true;
    }
  }

  // evaluasi pola C1 {delay + loss tanpa cpu}
  let c1Valid = false;
  if (hasR2 && hasR3 && !c2Valid) {
    const timestamps = [...r2Events, ...r3Events]
      .map(e => new Date(e.startedAt).getTime());
    const range = Math.max(...timestamps) - Math.min(...timestamps);
    // jika masih ddalam window - pola terpenuhi
    if (range <= CONFIG.correlationWindowMs) {
      c1Valid = true;
    }
  }

  if (!c2Valid) {
    if (!'indicationC2') { // buat indication C2 jika belum ada c2 yang active
      await Indication.create({ /* data indication */ });
    } else { // jika sudah ada, update indication c2
```

Source Code 17. Inisialisasi *correlation engine*

```
    await indicationC2.update({ lastSeenAt:new Date() });
  }

  if (indicationC1) { // jika sebelumnya ada C1, hapus C1 karena kalah prioritas
    await indicationC1.destroy();
  }

  } else if (!c1Valid) {
    if (!indicationC1) { // buat indication C1 jika belum ada c1 yang active
      await Indication.create({ /* data indication */ });
    } else { // jika sudah ada, update indication c1
      await indicationC1.update({ lastSeenAt:new Date() });
    }
  }

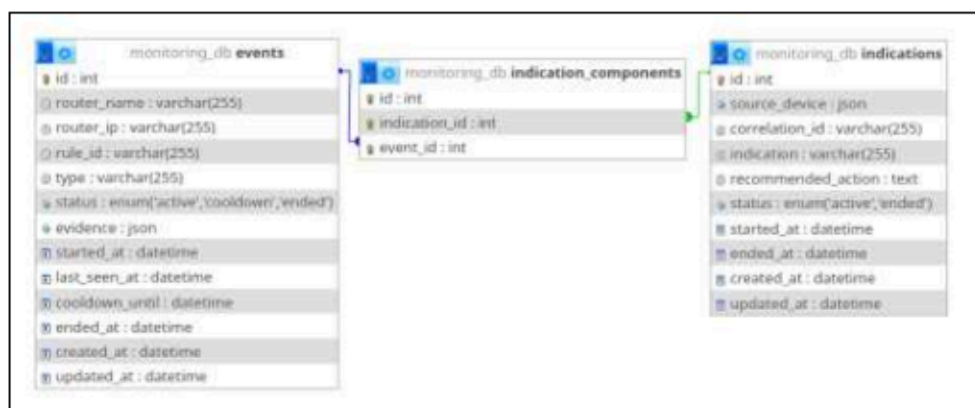
  if (indicationC2) { // jika sebelumnya ada C2, hapus karena tidak lagi valid
    await indicationC2.destroy();
  }
}

// mengakhiri indication jika semua event selesai
const activeIndications = /* ambil indication aktif */;
for (const ind of activeIndications) {
  const evs = /* ambil event terkait */;
  if (evs.length && evs.every(e => e.status === 'ended')) {
    await ind.update({ status:'ended', endedAt:new Date() });
  }
}
}
```

4.1.6 Implementasi Arsitektur Sistem

Implementasi arsitektur sistem dilakukan berdasarkan perancangan pada Bab III dengan mengintegrasikan proses pengambilan data dari perangkat MikroTik, pengolahan data, serta analisis menggunakan *rule-based detection* untuk pembentukan *event* dan *event correlation* untuk menghasilkan indikasi gangguan jaringan.

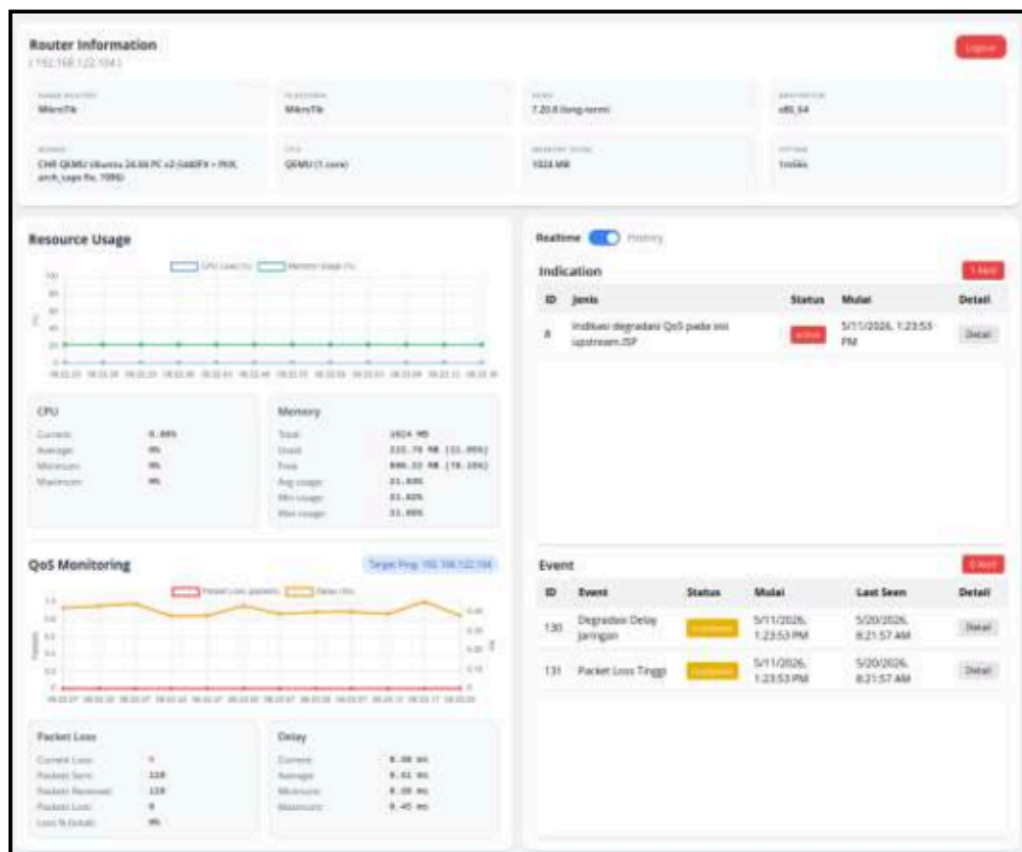
Implementasi basis data mengacu pada Entity Relationship Diagram (ERD) yang telah dirancang dan direalisasikan menggunakan MySQL. Struktur basis data berfokus pada penyimpanan data *event*, indikasi gangguan jaringan, serta relasi antara indikasi dan *event* pembentuknya. Hasil implementasi struktur basis data ditunjukkan pada Gambar 4.7.



Gambar 4.7 Implementasi Struktur Basis Data (Designer MySQL)

Implementasi antarmuka pengguna dilakukan berdasarkan wireframe yang telah dirancang dalam bentuk aplikasi web yang terdiri dari halaman login dan dashboard monitoring. Halaman login digunakan untuk menentukan perangkat yang akan dimonitor sekaligus sebagai autentikasi untuk memastikan keamanan koneksi ke perangkat tersebut. Setelah proses login berhasil dan sistem terhubung ke perangkat yang dimonitor, dashboard menampilkan hasil monitoring berupa statistik parameter jaringan serta informasi *event* dan indikasi gangguan. Tampilan antarmuka ditunjukkan pada Gambar 4.8 dan Gambar 4.9.

Gambar 4.8 Tampilan Halaman Login



Gambar 4.9 Tampilan Dashboard Monitoring

ID	Name	Type	Status
1	ether1	ether	Up
2	ether2	ether	Up
3	ether3	ether	Up
4	ether4	ether	Up
5	ether5	ether	Down
6	ether6	ether	Down
7	ether7	ether	Down
8	ether8	ether	Down
9	lo	loopback	Up

Gambar 4.10 Tampilan Dashboard Monitoring (Lanjutan)

4.2 Hasil

Bagian ini membahas hasil pengujian terhadap implementasi sistem yang telah dikembangkan. Pengujian yang dilakukan meliputi pengujian mekanisme deteksi dan pengujian sistem secara keseluruhan. Hasil pengujian mekanisme deteksi digunakan sebagai dasar evaluasi kinerja mekanisme deteksi menggunakan metrik *Mean Time to Detect* (MTTD) dan *Detection Rate* (DR). Pembahasan hasil pengujian disajikan pada subbab berikut.

4.2.1 Hasil Pengujian Mekanisme Deteksi

Pengujian mekanisme deteksi dilakukan sesuai dengan rancangan skenario pengujian yang telah dirancang sebelumnya. Setiap skenario diuji sebanyak tiga kali untuk memastikan konsistensi dan keandalan hasil deteksi. Hasil dari masing-masing pengujian disajikan pada Tabel 4.1.

Tabel 4.1 Hasil Pengujian Mekanisme Deteksi

Unit Uji	ID	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian		
				Ke-1	Ke-2	Ke-3
R1	S1	Menonaktifkan interface secara manual pada CHR	Event Interface Down terbentuk beserta evidence kondisi saat pembentukan	Sukses	Sukses	Sukses
	S2	Mengaktifkan kembali interface	Status event berubah menjadi ended	Sukses	Sukses	Sukses
R2	S3	Meningkatkan delay menggunakan tc netem sebesar 250ms dan mempertahankannya selama 60 detik	Event Delay Tinggi terbentuk beserta evidence kondisi saat pembentukan	Sukses	Sukses	Sukses
	S4	Mengembalikan delay ke kondisi normal hingga melewati durasi cooldown	Status event berubah menjadi cooldown, kemudian menjadi ended setelah durasi cooldown terpenuhi	Sukses	Sukses	Sukses
	S5	Meningkatkan kembali delay hingga melewati threshold sebelum durasi cooldown berakhir	Status event kembali menjadi active dan kejadian tetap dianggap satu event	Sukses	Sukses	Sukses

Tabel 4.2 Hasil Pengujian Mekanisme Deteksi (Lanjutan)

Unit Uji	ID	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian		
				Ke-1	Ke-2	Ke-3
R2	S6	Menjaga kondisi delay dalam batas normal (tidak melebihi threshold)	Event tidak terbentuk	Sukses	Sukses	Sukses
R3	S7	Menginjeksi packet loss menggunakan tc sebesar 5% dan mempertahankannya selama 60 detik	Event Packet Loss Tinggi terbentuk beserta evidence kondisi saat pembentukan	Sukses	Sukses	Sukses
	S8	Menghentikan packet loss hingga melewati durasi cooldown	Status event berubah menjadi cooldown, kemudian menjadi ended setelah durasi cool down terpenuhi	Sukses	Sukses	Sukses
	S9	Memunculkan kembali packet loss hingga melewati threshold sebelum durasi cooldown berakhir	Status event kembali menjadi active dan kejadian tetap dianggap satu event	Sukses	Sukses	Sukses
	S10	Menjaga packet loss dalam kondisi normal (tidak melebihi threshold)	Event tidak terbentuk	Sukses	Sukses	Sukses
R4	S11	Meningkatkan beban CPU menggunakan hping3 hingga sebesar $\pm 90-100\%$ dan dipertahankan selama 60 detik	Event Beban CPU Berlebih terbentuk beserta evidence kondisi saat pembentukan	Sukses	Sukses	Sukses
	S12	Menghentikan beban trafik hingga CPU stabil melewati durasi cool down	Status event berubah menjadi cooldown, kemudian menjadi ended setelah durasi cool down terpenuhi	Sukses	Sukses	Sukses
	S13	Meningkatkan kembali beban CPU hingga melewati threshold sebelum durasi cooldown berakhir	Status event kembali menjadi active dan kejadian tetap dianggap satu event	Sukses	Sukses	Sukses
	S14	Menjaga beban CPU dalam kondisi normal (tidak melebihi threshold)	Event tidak terbentuk	Sukses	Sukses	Sukses
C1	S15	Membentuk event QoS tanpa event CPU dalam rentang waktu korelasi	Indikasi degradasi kualitas jaringan tanpa keterlibatan resource perangkat (C1) terbentuk dan tersimpan	Sukses	Sukses	Sukses
	S16	Mengakhiri kondisi QoS (delay dan packet loss)	Status indikasi berubah menjadi ended dan waktu selesai indikasi tercatat	Sukses	Sukses	Sukses
	S17	Meningkatkan hanya salah satu parameter QoS (delay atau packet loss) hingga membentuk event	Indikasi C1 tidak terbentuk	Sukses	Sukses	Sukses

Tabel 4.3 Hasil Pengujian Mekanisme Deteksi (Lanjutan)

Unit Uji	ID	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian		
				Ke-1	Ke-2	Ke-3
C2	S18	Membentuk event QoS dan event CPU dalam rentang waktu korelasi	Indikasi degradasi kualitas jaringan akibat keterbatasan resource perangkat (C2) terbentuk dan tersimpan	Sukses	Sukses	Sukses
	S19	Mengakhiri event QoS dan CPU	Status indikasi berubah menjadi ended dan waktu selesai indikasi tercatat	Sukses	Sukses	Sukses
	S20	Membentuk event CPU di luar rentang waktu korelasi dari event QoS	Indikasi C2 tidak terbentuk, sistem hanya mempertahankan indikasi C1	Sukses	Sukses	Sukses

Data hasil pengujian dari Tabel 4.1 dan Tabel 4.2 kemudian dilakukan perhitungan selisih waktu antara dimulainya gangguan dan waktu sistem mendeteksi gangguan, yang hanya diterapkan pada pengujian dengan event atau indikasi gangguan pada masing-masing unit uji, yaitu S1, S3, S7, S11, S14, dan S17. Waktu awal gangguan dicatat secara manual menggunakan stopwatch pada perangkat eksternal, sedangkan waktu deteksi diperoleh dari timestamp saat event atau indikasi terbentuk pada sistem monitoring, kemudian selisih waktu dihitung sebagai perbedaan antara kedua waktu tersebut dan hasilnya disajikan pada Tabel 4.3.

Tabel 4.4 Hasil Perhitungan Selisih Waktu Deteksi

ID Pengujian	Deteksi yang Dihasilkan	Waktu Gangguan Dimulai	Waktu Gangguan Terdeteksi	Selisih Waktu (detik)
S1	Event (R1)	10.02.00	10.02.06	6
		10.03.00	10.03.05	5
		10.04.00	10.04.06	6
S3	Event (R2)	10.10.00	10.11.06	66
		10.15.00	10.16.05	66
		10.20.00	10.21.01	61
S7	Event (R3)	11.15.00	11.15.40	40
		11.20.00	11.21.10	71
		11.25.00	11.25.56	56
S11	Event (R4)	11.40.00	11.41.01	61
		11.45.00	11.46.01	61
		11.50.00	11.51.06	66
S15	Indikasi Gangguan (C1)	12.10.00	12.11.01	61
		12.15.00	12.16.06	66
		12.20.00	12.21.06	66
S18	Indikasi Gangguan (C2)	12.40.00	12.41.11	71
		12.45.00	12.46.06	66
		12.50.00	12.51.11	71

Berdasarkan data pada Tabel 4.3, dilakukan perhitungan nilai *Mean Time To Detect* (MTTD) dari mekanisme deteksi yang dirancang menggunakan Persamaan (2.1). Perhitungan dilakukan pada tiga tingkat, yaitu untuk setiap unit pengujian (R1, R2, R3, R4, C1, dan C2), berdasarkan mekanisme yang digunakan (*rule engine* dan *correlation engine*), serta secara keseluruhan sistem. Perhitungan MTTD pada setiap unit pengujian ditunjukkan sebagai berikut:

$$MTTD_{(R1)} = \frac{6 + 5 + 6}{3} = 5.67 \text{ detik}$$

$$MTTD_{(R2)} = \frac{66 + 66 + 61}{3} = 64.33 \text{ detik}$$

$$MTTD_{(R3)} = \frac{40 + 71 + 56}{3} = 55.67 \text{ detik}$$

$$MTTD_{(R4)} = \frac{61 + 61 + 66}{3} = 62.67 \text{ detik}$$

$$MTTD_{(C1)} = \frac{61 + 66 + 66}{3} = 64.33 \text{ detik}$$

$$MTTD_{(C2)} = \frac{71 + 66 + 71}{3} = 69.33 \text{ detik}$$

Perhitungan MTTD juga dilakukan berdasarkan mekanisme yang digunakan, yaitu event yang dihasilkan oleh *rule engine* dan indikasi yang dihasilkan oleh *correlation engine*. Event interface (R1) tidak disertakan dalam kelompok event karena memiliki karakteristik deteksi yang bersifat instan dan tidak melalui proses *sliding window* maupun korelasi, sehingga tidak merepresentasikan mekanisme deteksi berbasis QoS dan resource. Perhitungan MTTD pada setiap mekanisme deteksi ditunjukkan sebagai berikut:

$$MTTD_{(event)} = \frac{66 + 66 + 61 + 40 + 71 + 56 + 61 + 61 + 66}{9} = 60.89 \text{ detik}$$

$$MTTD_{(indikast)} = \frac{61 + 66 + 66 + 71 + 66 + 71}{6} = 66.83 \text{ detik}$$

MTTD secara keseluruhan dihitung dari seluruh data pengujian sebagai representasi umum performa sistem dalam mendeteksi gangguan ditunjukkan sebagai berikut:

$$MTTD_{(keseluruhan)} = \frac{909}{18} = 50.5 \text{ detik}$$

Selanjutnya, data hasil pengujian pada Tabel 4.1 digunakan untuk menghitung nilai *Detection Rate* (DR), yaitu dengan membandingkan jumlah insiden yang berhasil terdeteksi dengan jumlah insiden yang diujikan. Berdasarkan hasil pengujian, jumlah insiden yang berhasil terdeteksi dengan benar adalah sebanyak 18 dari total 18 insiden yang diujikan. Oleh karena itu, nilai *Detection Rate* dihitung menggunakan Persamaan (2.2) sebagai berikut:

$$DR = \frac{18}{18} \times 100\% = 100\%$$

4.2.2 Hasil Pengujian Sistem

Tahap pengujian sistem dilakukan sesuai dengan rancangan skenario uji *blackbox* yang telah dirancang untuk mengetahui apakah sistem yang dikembangkan mampu memenuhi kebutuhan fungsionalnya. Hasil pengujian *blackbox* ditunjukkan pada Tabel 4.4.

Tabel 4.5 Hasil Pengujian *Blackbox*

No.	Fitur	Skenario Pengujian	Hasil yang Diharapkan	Hasil
1.	Autentikasi dan koneksi perangkat	Login dengan data yang valid.	Autentikasi berhasil dan sistem terhubung dengan perangkat MikroTik.	Sukses
		Login dengan data yang tidak valid.	Login gagal dengan pesan kesalahan kredensial.	Sukses
		Data login valid tetapi perangkat MikroTik tidak dapat dijangkau.	Login gagal dengan pesan koneksi gagal.	Sukses
2.	Pengambilan parameter jaringan	Sistem menjalankan proses polling parameter kesehatan jaringan.	Parameter kesehatan jaringan diterima dan tampil di dashboard	Sukses
3.	Pembentukan event gangguan	Kondisi salah satu parameter memenuhi rule.	Event beserta evidence terbentuk dan berhasil ditampilkan.	Sukses
4.	Pembentukan indikasi gangguan	Beberapa event yang berkaitan muncul dalam window korelasi	Indikasi beserta event pembentuknya terbentuk dan berhasil ditampilkan.	Sukses
5.	Penyimpanan hasil analisis	Event atau indikasi gangguan terbentuk	Data event, indikasi, dan relasi keduanya tersimpan di database	Sukses

4.3 Pembahasan

Hasil pengujian skenario gangguan pada lingkungan uji yang telah dirancang dengan satu router utama sebagai objek monitoring menunjukkan bahwa sistem mampu membentuk dan mengaitkan beberapa *event* yang muncul dalam rentang waktu berdekatan menjadi indikasi awal gangguan yang lebih kontekstual. Pendekatan *rule-based* pada *rule engine* membentuk *event* berdasarkan *threshold* dan durasi pelanggaran melalui *sliding window*, sedangkan *event correlation* berbasis *temporal similarity* pada *correlation engine* mengaitkan *event* berdasarkan kedekatan waktu dan keterkaitan parameter. Selain itu, mekanisme *event lifecycle* memastikan bahwa *event* hanya diakhiri ketika kondisi jaringan telah kembali normal secara stabil.

Berdasarkan evaluasi terhadap hasil pengujian, sistem *monitoring* yang dikembangkan menunjukkan nilai *Detection Rate* (DR) sebesar 100%, yang menandakan seluruh skenario gangguan berhasil terdeteksi. Secara teoretis, capaian ini dipengaruhi oleh sifat deterministik dari pendekatan *rule-based* dan *event correlation* berbasis *temporal similarity*. Pada pendekatan *rule-based*, kondisi gangguan direpresentasikan melalui aturan eksplisit yang membentuk *event* berdasarkan perbandingan nilai parameter terhadap *threshold* serta pemenuhan durasi pelanggaran tertentu. Pendekatan *event correlation* menggunakan prinsip serupa dengan mengaitkan *event* berdasarkan kedekatan waktu kemunculan dan pola hubungan antar parameter. Karena kedua mekanisme bergantung pada aturan yang telah didefinisikan, sistem akan menghasilkan *output* yang konsisten untuk kondisi *input* yang sama.

Hasil evaluasi juga menunjukkan bahwa nilai *Mean Time to Detect* (MTTD) secara keseluruhan sebesar 50,5 detik. Pada mekanisme *rule engine*, MTTD untuk kelompok event (R2, R3, dan R4) sebesar 60,89 detik dengan variasi sekitar 5–6 detik pada setiap pengujian. Nilai ini dipengaruhi oleh aturan durasi pelanggaran selama 60 detik pada *sliding window*, sehingga event baru terbentuk setelah kondisi gangguan bertahan dalam periode tersebut. Proses evaluasi yang dilakukan setiap 5 detik menyebabkan waktu deteksi tidak selalu tepat di 60 detik, melainkan dapat bergeser mengikuti siklus pengambilan data. Variasi waktu pada parameter *packet loss* cenderung lebih tidak stabil dibandingkan parameter lain karena mekanisme injeksi menggunakan *netem* yang bersifat probabilistik, serta perhitungan dalam *window* yang menggunakan total kehilangan paket.

Perbedaan karakteristik waktu deteksi juga terlihat pada mekanisme lain dalam sistem. Nilai MTTD pada event R1 (*interface*) jauh lebih rendah, yaitu sekitar 5,67 detik karena deteksi dilakukan berdasarkan perubahan status secara langsung tanpa melalui proses dalam *window*. Sebaliknya, pada mekanisme *correlation engine*, nilai MTTD lebih tinggi, yaitu sebesar 66,83 detik karena pembentukan indikasi memerlukan terpenuhinya beberapa event dalam rentang waktu tertentu. Penerapan *event lifecycle* seperti *cooldown* juga membatasi pembentukan event baru sehingga pada kondisi tertentu waktu deteksi dapat bertambah. Secara keseluruhan, perbedaan nilai MTTD ini menunjukkan bahwa waktu deteksi sangat dipengaruhi oleh desain aturan yang digunakan, khususnya durasi pelanggaran dan mekanisme evaluasi, sehingga sistem cenderung memprioritaskan konsistensi kondisi dibandingkan respons instan.

Secara keseluruhan, kombinasi *rule-based detection* dan *event correlation* menghasilkan sistem *monitoring* yang akurat sekaligus mampu menyusun indikasi awal gangguan secara terstruktur. Nilai *Detection Rate* yang tinggi menunjukkan efektivitas deteksi berbasis aturan, sementara MTTD mencerminkan adanya mekanisme kontrol untuk memastikan validitas deteksi, bukan sekadar respons instan. Hal ini menegaskan bahwa sistem menyeimbangkan keandalan dan kecepatan, sehingga mampu mengurangi kesalahan interpretasi dan mendukung pengambilan keputusan yang lebih tepat oleh administrator.

Sementara itu, pengujian sistem menggunakan metode *black-box testing* dilakukan untuk memastikan seluruh fungsionalitas berjalan sesuai rancangan, mulai dari pengambilan data perangkat MikroTik, pengolahan *sliding window*, pembentukan event oleh *rule engine*, hingga penyajian pada *dashboard*. Hasil pengujian menunjukkan bahwa seluruh fungsi

berjalan sesuai dengan yang diharapkan, sehingga alur monitoring dapat beroperasi tanpa kesalahan fungsional. Hal ini menegaskan bahwa seluruh mekanisme, termasuk *poller*, *rule engine*, dan *correlation engine*, telah diimplementasikan dengan baik.

BAB V PENUTUP

5.1 Kesimpulan

Berdasarkan hasil penelitian dan pengujian yang telah dilakukan, sistem monitoring kesehatan jaringan berbasis MikroTik berhasil diimplementasikan dan mampu mengolah serta menyajikan parameter kesehatan jaringan secara periodik melalui dashboard. Sistem juga berhasil menerapkan pendekatan *rule-based detection* dalam membentuk *event* gangguan berdasarkan parameter yang diamati, dengan memanfaatkan nilai ambang (*threshold*) dan durasi pengamatan. Selain itu, pendekatan *event correlation* berhasil diterapkan untuk menganalisis keterkaitan antar *event* sehingga mampu menghasilkan indikasi awal penyebab gangguan jaringan secara terstruktur.

Dari sisi kinerja, sistem menunjukkan nilai *Mean Time to Detect* (MTTD) sebesar 50.5 detik secara keseluruhan, dengan 60.89 detik untuk deteksi *event* dan 66.83 detik untuk indikasi gangguan. Selain itu, nilai *Detection Rate* (DR) mencapai 100%, yang menunjukkan bahwa seluruh insiden pada skenario pengujian berhasil terdeteksi dengan benar. Hasil ini menunjukkan bahwa sistem yang dikembangkan telah memenuhi tujuan penelitian serta mampu melakukan monitoring, deteksi, dan analisis gangguan jaringan secara konsisten.

5.2 Saran

Adapun saran untuk pengembangan penelitian selanjutnya adalah sebagai berikut :

1. Penelitian selanjutnya dapat menambahkan parameter kesehatan jaringan lain, seperti *throughput*, *jitter*, atau metrik lainnya agar analisis kondisi jaringan menjadi lebih komprehensif.
2. Penelitian selanjutnya dapat mengoptimalkan nilai *threshold*, durasi pelanggaran, dan pola korelasi agar sistem lebih adaptif dan efektif dalam mendeteksi indikasi gangguan jaringan.
3. Penelitian selanjutnya dapat melakukan pengujian pada beberapa router secara paralel untuk mengetahui kemampuan pendekatan yang digunakan dalam mendeteksi indikasi gangguan pada kondisi jaringan yang lebih kompleks.

DAFTAR PUSTAKA

- Alkenani, J., & Nassar, K. (2022). Network Monitoring Measurements for Quality of Service: A Review. *Al-Mağallāt al-‘irāqīyyāt al-Handasāt al-Kahrabā’iyyāt-al-Ilaktrūniyyāt*, 18(2), 33–42.
- Amin, M., Hasbi, M., Widada, B., & Yudarto, B. W. (2025). Implementasi Monitoring Jaringan Internet pada MikroTik Menggunakan Notifikasi Bot Telegram di SMK. *J. Teknol. Inf. Dan Komun. (TIKomSiN)*, 13(1), 67.
- Asosiasi Penyelenggara Jasa Internet Indonesia. (2025). *Profil Internet Indonesia 2025*.
- Baydowi, W., & Al Islami, H. (2026). Sistem Monitoring Jaringan Berbasis Web dengan Fitur Notifikasi Insiden Otomatis dan Pencatatan Log Historis pada PT. Mitra Bersama Jaya. *Jati*, 10(1), 641–645.
- Caicedo-Altamirano, F., Alpúsig-Cuichán, S., Villacrés-Cevallos, P., Nacimba, B. C., Loachamin-Alvarez, A., & Guamán-Yanez, M. (2025). Implementation of a Virtual Networking Laboratory for Corporate Network Simulation Using GNS3. *2025 IEEE Ninth Ecuador Technical Chapters Meeting(ETCM)*, 1–4.
- Caicedo-Altamirano, F., Villacrés-Cevallos, P., Alpusig-Cuichan, S., & Ramos-Jinez Alex and Guerra-Pazo, O. (2025). Cloud-Based Network Monitoring System with Zabbix and WireGuard VPN for Remote SMEs. *2025 IEEE Ninth Ecuador Technical Chapters Meeting(ETCM)*, 1–6.
- De Gasperis, G., & Facchini, S. D. (2025). A Comparative Study of Rule-Based and Data-Driven Approaches in Industrial Monitoring. *ArXiv (Cs.AI)*.
- ElSahly, O., & Abdelfatah, A. (2022). A Systematic Review of Traffic Incident Detection Algorithms. *Sustainability*, 14(22), 14859.
- Fachrurrozi, N. R., Wirabudi, A. A., & Rozano, S. A. (2023). Design of Network Monitoring System Based on LibreNMS Using Line Notify, Telegram, and Email Notification. *Sinergi*, 27(1), 111.
- Gomez, J., Kfoury, E. F., Crichigno, J., & Srivastava, G. (2023). A Survey on Network Simulators, Emulators, and Testbeds Used for Research and Education. *Comput. Netw.*, 237(110054), 110054.
- International Telecommunication Union. (2011). *ITU-T Y.1541: Network Performance Objectives for IP-Based Services* (Number Y.1541).
- International Telecommunication Union. (2019). *ITU-T Y.1540: Internet Protocol Data Communication Service – IP Packet Transfer and Availability Performance Parameters* (Number Y.1540).
- Irawansyah, R., Khairil, K., & Alinse, R. T. (2024). Designing a Computer Network Monitoring System with SMS Notification Using The Dude. *Jmcs*, 3(1).
- Kodors, S., Apeinans, I., Zarembo, I., & Lonska, J. (2025). RecGen: No-Coding Shell of Rule-Based Expert System with Digital Twin and Capability-Driven Approach Elements for Building Recommendation Systems. *Appl. Sci. (Basel)*, 15(19), 10482.
- Kotenko, I., Gaifulina, D., & Zelichenok, I. (2022). Systematic Literature Review of Security Event Correlation Methods. *IEEE Access*, 10, 43387–43420.

- Maestro Pasaribu, Y., & Ferdiansyah, F. (2024). Implementation of Network and Server Monitoring on Mikrotik with Netwatch via Telegram Bot. *Sitekin*, 21(2), 409.
- Malik, P. F., & Josaphat, B. P. (2024). Desain dan Implementasi Sistem Monitoring Jaringan Menggunakan Zabbix dan Telegram. *Semnasoffstat*, 2024(1), 711–722.
- MikroTik. (2026a). *Cloud Hosted Router - RouterOS Documentation*. <https://help.mikrotik.com/docs/display/ROS/Cloud+Hosted+Router>
- MikroTik. (2026b). *Dude - RouterOS Documentation*. <https://help.mikrotik.com/docs/spaces/ROS/pages/377225250/Dude>
- MikroTik. (2026c). *Netwatch - RouterOS Documentation*. <https://help.mikrotik.com/docs/display/ROS/Netwatch>
- MikroTik. (2026d). *RouterOS - MikroTik Documentation*. <https://help.mikrotik.com/docs/display/ROS/RouterOS>
- Pillo-Guanoluisa, D., Herrera Leandro Bermúdez and Alba, A. J., Báez, D. S., & Buestán, V. F. (2025). Real-Time Network Infrastructure Monitoring: A Zabbix and Telegram Integration Case Study in the Energy Sector (EMELNORTE S.A.). *2025 IEEE Ninth Ecuador Technical Chapters Meeting (ETCM)*, 1–6.
- Pratama, A. B. P., Handoyo, E., & Shodiq, M. (2025). Design and Implementation of The Dude MikroTik Server Using Telegram Bot as Monitoring Client Network and Internet Connection. *Indonesian Journal of Engineering, Science and Technology*, 2(1), 26–36.
- Rahayu, S. P., & Prisma, I. G. L. P. E. (2022). Implementasi Monitoring Manajemen Jaringan dengan Software The Dude Berbasis Telegram Messenger. *Journal of Informatics and Computer Science (JINACS)*, 4(01), 19–25.
- Syairozi, A. A., & Arizal. (2025). Comparative Analysis of eBPF-Based Runtime Security Monitoring Tools in Monitoring and Threat Detection on Kubernetes. *Proceedings of the 1st International Conference on Research and Innovations in Information and Engineering*, 136–141.
- Troia, S., Savi, M., Nava, G., Zorello, L. M. M., Schneider, T., & Maier, G. (2023). Performance Characterization and Profiling of Chained CPU-Bound Virtual Network Functions. *Comput. Netw.*, 231(109815), 109815.